FM Pocket Radio

Phase 3 — PCB & Product Build Guide

ATmega328P • RDA5807M • PAM8403 • CH340C • SSD1306 OLED

KiCad 9.0 • USB-C • Full Step-by-Step Build

EE2050 — Product Development Module

March 2026

Parameter	Value
Target Board Size	$\sim$70 mm $\times$ 50 mm
PCB Layers	2 (double-sided FR-4, SMD + through-hole)
USB-C	Charging **and** programming (CH340C)
MCU	ATmega328P @ 8 MHz, 3.3 V
Power	3.7 V LiPo (800–1000 mAh)
Audio Output	Speaker + 3.5 mm headphone jack
FM Range	87.5–108.0 MHz (Sri Lanka)

0 Contents

1 Functionality **5**

2 Project Overview **5**
2.1 Design Goals . . . 5
2.2 Key Decisions . . . 6

3 System Architecture **7**
3.1 Block Diagram . . . 7
3.2 Power Architecture . . . 7
3.3 DW01A + FS8205A Battery Protection (Low-Side) . . . 8
3.4 Audio Signal Path . . . 8

4 Complete Bill of Materials **9**
4.1 ICs and Modules . . . 9
4.2 Resistors . . . 10
4.3 Capacitors . . . 11
4.4 Connectors, Switches, and Mechanical . . . 11
4.5 Mechanical (Enclosure) . . . 12
4.6 Estimated Cost . . . 13

5 Complete Circuit Design **13**
5.1 ATmega328P Pin Assignment . . . 13
5.2 TP4056 LiPo Charger IC (SOP-8, On-Board) . . . 15
5.3 Power Switch and LDO . . . 15
5.4 RDA5807M Module Connections . . . 16
5.5 PAM8403 Class-D Amplifier (SOP-16, Stereo BTL) . . . 16
5.6 3.5 mm Headphone Jack . . . 17
5.7 I2C Bus . . . 17
5.8 Rotary Encoders . . . 17
5.9 Programming Interface (CH340C + USB-C) . . . 18
5.10 Battery Voltage Divider . . . 18
5.11 Power LED . . . 19
5.12 Complete Connection Checklist . . . 19

6 KiCad 9.0 — Schematic Entry (Step by Step) **21**
6.1 Opening the Project . . . 21
6.2 Setting Up Schematic Sheets . . . 22
6.3 Placing Components . . . 22
6.4 Wiring . . . 22
6.5 Annotation and Values . . . 23
6.6 Running ERC . . . 23

7 Importing Custom Components in KiCad 9.0 **23**
7.1 Method 1: Download from SnapEDA (Recommended) . . . 23
7.1.1 Register Symbol Library . . . 23
7.1.2 Register Footprint Library . . . 24
7.2 Method 2: Download from Ultra Librarian . . . 24
7.3 Method 3: Create Custom Symbols Manually . . . 24
7.3.1 Creating a Symbol for the RDA5807M Module (10-pin) . . . 24
7.3.2 Creating a Footprint for the RDA5807M Module . . . 24

7.3.3 Creating a Symbol/Footprint for the SSD1306 OLED Module 25

8 KiCad 9.0 — PCB Layout (Step by Step) 25
8.1 Transfer Schematic to PCB 25
8.2 Assign Footprints 25
8.3 Set Board Outline 26
8.4 Component Placement Guidelines 26
8.5 PCB Component Layout Diagram (Top View) 27
8.6 Noise Reduction and Signal Integrity 27
8.7 Design Rules Setup 28
8.8 Routing Traces 28
8.9 Ground Plane Pour 28
8.10 Silkscreen 29
8.11 Running DRC 29

9 Generating Gerber Files 29
9.1 BOM and Pick-and-Place Export (If Shop Does Assembly) 30
9.2 Sending to PCB Shop 30

10 Enclosure Design — Laser-Cut Acrylic 30
10.1 Concept: Stacked Acrylic Layers 30
10.2 Layer Drawings (Dimensions in mm) 30
10.3 Design in Fusion 360 31
10.4 Assembly 31

11 Firmware 31
11.1 Arduino IDE Setup 31
11.2 Required Libraries 32
11.3 Complete Firmware Source Code 32

12 Programming the ATmega328P 39
12.1 Step 1: Burn Bootloader (One-Time Only) 39
12.2 Step 2: Upload Firmware (Repeatable) 40
12.3 Troubleshooting Programming 40

13 Assembly Guide 41
13.1 If PCB Shop Pre-Solders 41
13.2 If Hand-Soldering Everything 42
13.3 Module Installation 42
13.4 Final Assembly 42

14 Testing & Verification 42
14.1 Test 1: Visual Inspection 42
14.2 Test 2: Power Supply (Before Inserting ICs) 43
14.3 Test 3: TP4056 Charging via USB-C 43
14.4 Test 4: MCU Boot 43
14.5 Test 5: I2C Scan 43
14.6 Test 6: FM Radio Reception 44
14.7 Test 7: Speaker Output 44
14.8 Test 8: Headphone Auto-Mute 44
14.9 Test 9: EEPROM Persistence 44
14.10 Test 10: Battery Level 45

15 Safety Precautions **45**

A ATmega328P-PU (DIP-28) Full Pinout **45**

B RDA5807M Key Registers **46**

C KiCad Custom Library Quick Reference **46**

1 Functionality

Use this checklist to verify every feature works after assembly. Tick each box during testing (Section 14).

#	Feature
☐	**Power on/off** via latching push button (SW1). Press once = ON, press again = OFF.
☐	**Charge via USB-C**: plug any 5 V charger into the single USB-C port. Red LED = charging; green LED = full. Radio can be used while charging (but charge termination may be delayed — see Section 12 caveat).
☐	**Upload firmware via same USB-C**: in Arduino IDE select "Arduino Pro Mini, 3.3 V, 8 MHz", choose the COM port. Click Upload. No external dongle needed.
☐	**Burn bootloader** (one-time only): connect USBasp or Arduino-as-ISP to the 6-pin ISP header (J3).
☐	**OLED display** shows: frequency (e.g. 98.9), station name from built-in frequency database, volume bar (0–15), signal strength bars, battery percentage, HP/SPK icon.
☐	**Right encoder**: rotate = tune $\pm$0.1 MHz; press = auto-seek UP to next strong station.
☐	**Left encoder**: rotate = volume 0–15; press = auto-seek DOWN to previous station.
☐	**Headphone auto-detect**: plug headphones in $\rightarrow$ speaker mutes (PAM8403 ~SHDN pulled LOW), audio routes to headphones. Unplug $\rightarrow$ speaker resumes.
☐	**Headphone cable = FM antenna**: the headphone wire couples to the RDA5807M ANT pin via C8, improving reception.
☐	**Battery monitoring**: voltage divider on A0 reads LiPo voltage; displayed as percentage on OLED.
☐	**Boot screen**: "FM RADIO" + "EE2050" shown for 2 s on power-up, then main screen.
☐	**EEPROM persistence**: last tuned frequency + volume saved to EEPROM; restored on next power-on.
☐	**Power LED**: red 3 mm LED lights when radio is ON (indicates +3.3 V rail active).

2 Project Overview

This document is a complete, step-by-step guide for designing a pocket-size FM radio as a custom PCB product. It covers circuit design, KiCad schematic entry, PCB layout, Gerber file generation, enclosure design, firmware, programming, assembly, and testing.

2.1 Design Goals

- **Pocket-size:** PCB $\sim$70$\times$50 mm, total thickness $<$25 mm
- **Mixed SMD + through-hole:** SMD ICs (PAM8403, CH340C, TP4056, HT7333) soldered by

university workshop. Through-hole parts (ATmega, crystal, connectors) soldered by user.

- **2-layer PCB:** Top = components + signal traces. Bottom = continuous ground plane for noise isolation.
- **Single USB-C port:** Handles **both** LiPo charging (via TP4056) and firmware upload (via CH340C). One cable for everything.
- **Product-grade finish:** Laser-cut acrylic enclosure, knob caps, clean ports
- **Full FM radio:** 87.5–108.0 MHz with station name display (built-in frequency database)
- **Dual output:** Built-in speaker **and** headphone jack with auto-switching
- **Rechargeable:** LiPo battery, 6–8 hr playback
- **Low noise:** Analog/digital ground separation, Class-D amplifier, proper decoupling
- **Budget:** Rs. 2500–5000 total

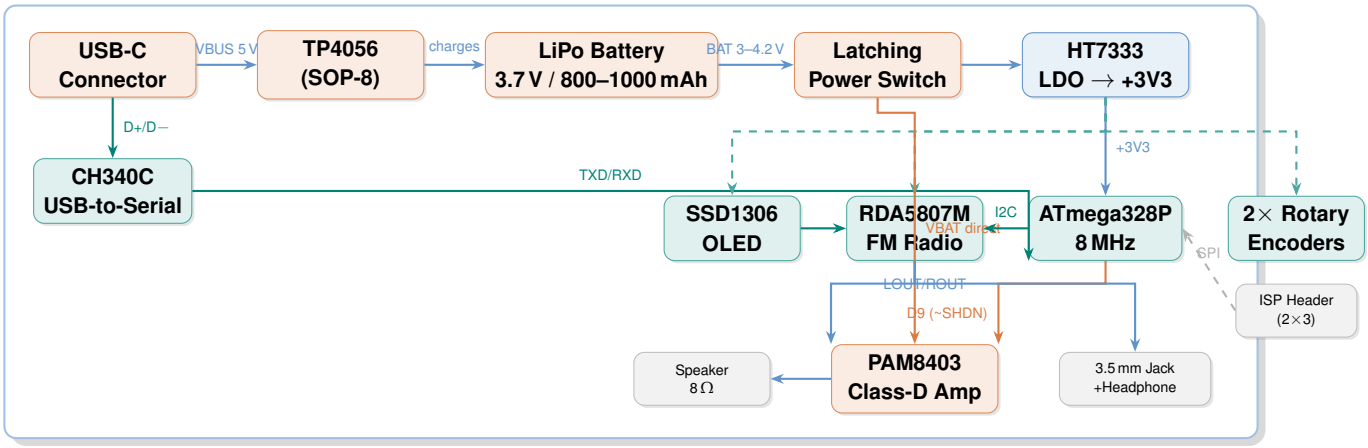
2.2 Key Decisions

Decision	Choice	Reason
MCU	ATmega328P-PU (DIP-28)	Same chip as Arduino UNO. 3.3 V / 8 MHz. Through-hole.
Radio IC	RDA5807M on RRD-102 V2.0 module	Already owned. Solders to 2×5 pin header.
Display	0.96" SSD1306 OLED 128×64	Already owned. Plugs into 4-pin header.
Amplifier	**PAM8403** (SOP-16)	Class-D stereo amplifier IC, used in **mono mode** (left channel only). ∼0.5 W into 8 Ω at 3.7 V VBAT. 90% efficient. ~SHDN pin for MCU-controlled mute. SMD — workshop solders.
Speaker mute	MCU-controlled (D9 → PAM8403 ~SHDN) + PJ-320A jack detect	D8 senses HP insertion; firmware pulls D9 LOW to mute amp.
Speaker	8 Ω, 0.5–1 W, 28–40 mm	Small, pocket-appropriate.
Encoders	2× rotary encoders + knob caps	Left = volume, Right = tuning.
Battery	Flat LiPo 800–1000 mAh	6–8 hr playback. Pocket-size.
Charging	**TP4056 IC** (SOP-8) on main PCB	Shares the single USB-C port. CHG + STDBY LEDs on PCB. Workshop solders.
LDO	**HT7333** (SOT-89)	3.3 V from VBAT. Low dropout (<100 mV). Workshop solders.
USB-to-serial	**CH340C** (SOP-16)	Permanent USB-C programming. No external dongle. Workshop solders.
USB connector	**USB-C** (SMD)	Single port: charging + programming. Workshop solders.
Audio output	3.5 mm jack + speaker	Headphone wire doubles as FM antenna.

Decision	Choice	Reason
Power switch	Latching push button	On VBAT line (after TP4056 OUT+), through enclosure.
Programming	USB-C (via CH340C) + ISP header	USB-C for daily uploads; ISP for one-time bootloader.
PCB	**2-layer** FR-4, ∼70×50 mm	Top = signals + components. Bottom = ground plane.
Enclosure	Clear acrylic, laser-cut layers	Display and LED visible through acrylic.
Volume	RDA5807M software (16 steps)	No potentiometer needed.

3 System Architecture

3.1 Block Diagram



3.2 Power Architecture

NOTE

The ATmega328P runs at **3.3 V and 8 MHz**. This is within the official specification (datasheet: 2.4–5.5 V for ≤10 MHz). Running at 3.3 V means:

- **No level shifters needed** — MCU, radio, and OLED all share the same 3.3 V rail
- **Lower power consumption** — longer battery life
- **Use an 8 MHz crystal** (not 16 MHz)
- Bootloader: "Arduino Pro Mini 3.3 V 8 MHz" in Arduino IDE

- **USB-C VBUS (5 V)** → TP4056 IN+ → charges LiPo (4.2 V CC/CV)
- **USB-C D+/D−** → CH340C → ATmega UART (firmware upload)
- **LiPo BAT+** → TP4056 BAT (pin 3) → latching switch → **VBAT** bus (3.0–4.2 V)
- **LiPo BAT−** → DW01A/FS8205A protection (low-side, on BAT− path) → GND bus. The DW01A monitors cell voltage; the FS8205A dual N-FET pair sits between BAT− and circuit GND, disconnecting the ground return on over-discharge, over-charge, or short-circuit.
- **VBAT** → HT7333 LDO (<100 mV dropout) → **+3V3** rail (MCU + radio + OLED + encoders + CH340C)
- **VBAT** → PAM8403 VDD directly (no LDO — maximises audio power; PAM8403 accepts 2.5–5.5 V)

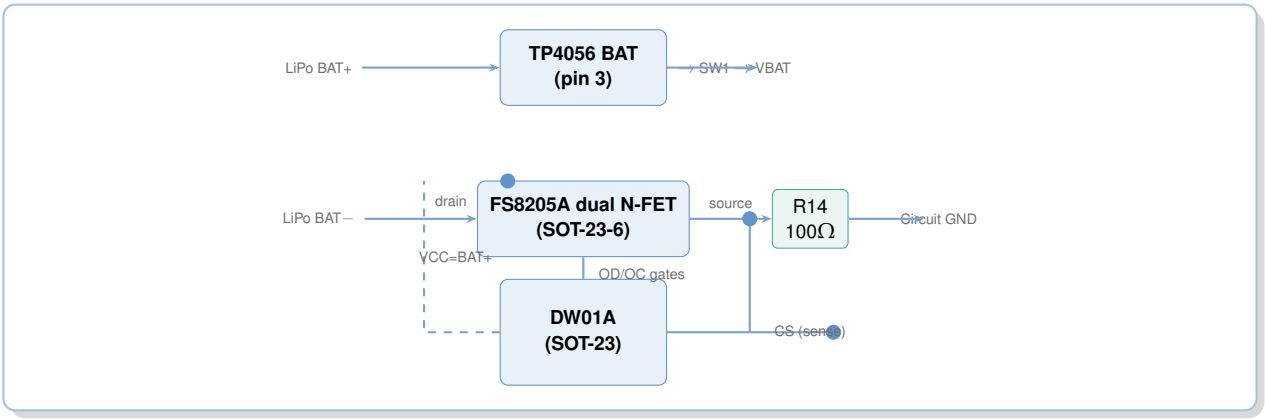
- **VBAT** → 100 k / 100 k voltage divider → ATmega A0 (battery sensing)

WARNING

The HT7333 LDO has <100 mV dropout. At battery minimum (3.0 V), output is ~2.9 V. The ATmega328P runs reliably at 2.9 V / 8 MHz (min spec 2.4 V).
Do NOT use AMS1117-3.3 — it requires 1 V headroom and will fail below 4.3 V battery.

3.3 DW01A + FS8205A Battery Protection (Low-Side)

The DW01A + FS8205A form a **low-side** battery protection circuit on the BAT− path. The FS8205A contains two N-channel MOSFETs in a single SOT-23-6 package. Because they are N-FETs, they **must** sit on the low side (between BAT− and circuit GND), not on BAT+.

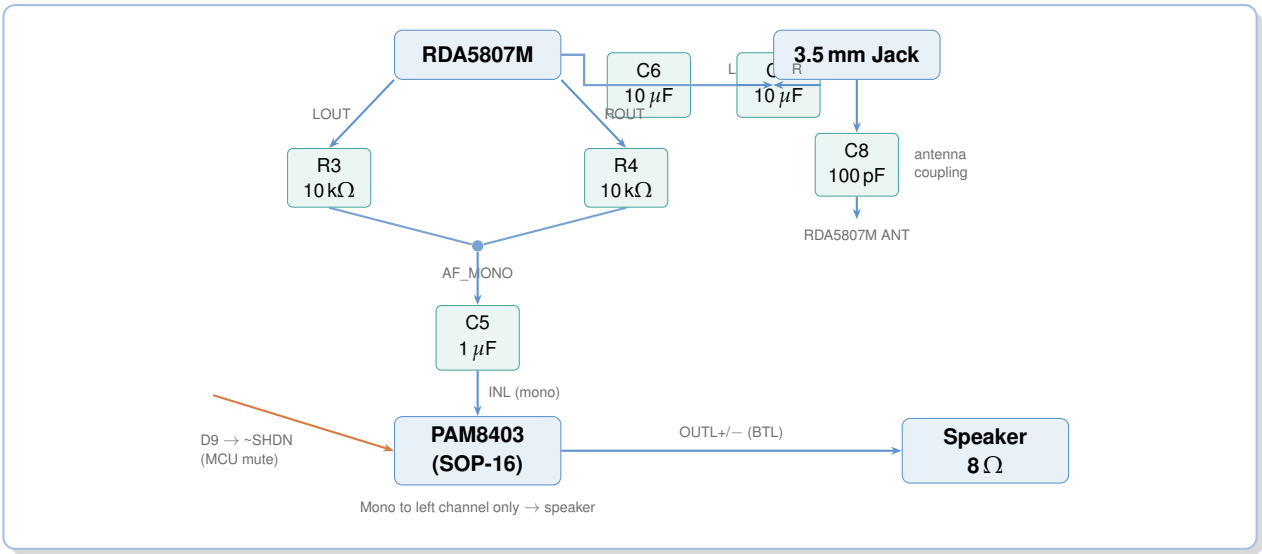


NOTE

DW01A + FS8205A wiring summary:

- **DW01A VCC** ← LiPo BAT+ (monitors cell voltage directly)
- **DW01A GND** → FS8205A source / R14 junction
- **DW01A OD** (over-discharge gate) → FS8205A gate 1
- **DW01A OC** (over-charge gate) → FS8205A gate 2
- **DW01A CS** (current sense) ← R14/FS8205A source junction
- **FS8205A drain** ← LiPo BAT−
- **FS8205A source** → R14 (100 Ω) → circuit GND bus
- **R14** senses over-current conditions (short-circuit protection)

3.4 Audio Signal Path



Volume is controlled digitally via RDA5807M register 0x05 bits [3:0], range 0–15, adjusted by the left rotary encoder. No hardware potentiometer.

NOTE

Audio topology — mono speaker, stereo headphones: The RDA5807M outputs stereo on LOUT/ROUT. For the **speaker path**, R3 and R4 (each 10 kΩ) passively sum LOUT + ROUT into a single mono signal (AF_MONO). This feeds through a single coupling capacitor C5 (1 µF) into PAM8403 **INL (pin 8) only**. INR (pin 7) is tied to GND via a 10 kΩ resistor to prevent noise pickup. Only the left BTL channel (OUTL+/OUTL−) drives the speaker. For the **headphone path**, LOUT and ROUT connect directly (via C6 and C7) to the 3.5 mm jack TIP and RING, preserving full stereo.

When headphones are plugged in, the MCU detects the insertion via D8 (HP_DETECT, jack pin 2) and pulls D9 LOW, which drives the PAM8403 ~SHDN pin LOW to mute the amplifier. Audio is routed to headphones via the coupling capacitors C6/C7. When headphones are removed, D9 goes HIGH and the speaker resumes.

4 Complete Bill of Materials

4.1 ICs and Modules

Ref	Part	Package	Qty	Notes
U1	ATmega328P-PU	DIP-28	1	Main MCU. Through-hole. Same as Arduino UNO chip.
U2	RDA5807M (RRD-102 V2.0)	Module	1	**Already owned.** Solders onto 2×5 pin header pads.
U3	**PAM8403**	SOP-16	1	Class-D amplifier. Wired mono (left channel only to 8 Ω speaker). ~0.5 W at 3.7 V into 8 Ω. ~SHDN pin for MCU mute. **SMD — workshop solders.**

Ref	Part	Package	Qty	Notes
U4	**TP4056**	SOP-8	1	LiPo CC/CV charger IC. On main PCB sharing USB-C VBUS. **SMD — workshop solders.**
U5	**HT7333**	SOT-89	1	3.3 V LDO regulator, 250 mA, low dropout (<100 mV). **SMD — workshop solders.**
U6	**CH340C**	SOP-16	1	USB-to-serial bridge. Connects USB-C D+/D- to ATmega UART. No external crystal needed. **SMD — workshop solders.**
U7	DW01A + FS8205A	SOT-23 + SOT-23-6	1+1	Battery protection: over-discharge, over-charge, short-circuit. **SMD — workshop solders.**
—	SSD1306 0.96″ OLED	Module	1	**Already owned.** 4-pin I2C. Plugs into header.

4.2 Resistors

Ref	Value	Qty	Purpose
R1	4.7 kΩ	1	I2C SDA pull-up to +3V3
R2	4.7 kΩ	1	I2C SCL pull-up to +3V3
R3	10 kΩ	1	RDA5807M LOUT → mono mix resistor (summed with R4 to AF_MONO)
R4	10 kΩ	1	RDA5807M ROUT → mono mix resistor (summed with R3 to AF_MONO)
R5	10 kΩ	1	ATmega RESET pull-up to +3V3
R6	330 Ω	1	Power LED current limiter (from +3V3)
R7	100 kΩ	1	Battery divider — top (VBAT to ADC)
R8	100 kΩ	1	Battery divider — bottom (ADC to GND)
R9	2.2 kΩ	1	TP4056 PROG pin — sets charge current to ∼455 mA ($I_{charge} = 1000/R9$). For 800 mAh cell: ∼0.57C. Charges in ∼2 hrs. Use 10 kΩ for 100 mA (very slow), 1.2 kΩ for 1 A (check USB source).
R10	5.1 kΩ	1	USB-C CC1 pull-down (tells charger “I am a device”)
R11	5.1 kΩ	1	USB-C CC2 pull-down
R12	1 kΩ	1	CHG LED current limiter: VBUS → R12 → LED2 → TP4056 CHRG pin 7 (open-drain)
R13	1 kΩ	1	STDBY LED current limiter: VBUS → R13 → LED3 → TP4056 STDBY pin 6 (open-drain)

Ref	Value	Qty	Purpose
R14	100 Ω	1	DW01A current sense resistor between FS8205A source pins and circuit GND (low-side topology)

4.3 Capacitors

Ref	Value	Type	Qty	Purpose
C1	100 nF	Ceramic	1	ATmega VCC decoupling
C2	100 nF	Ceramic	1	ATmega AVCC decoupling
C3	22 pF	Ceramic	2	Crystal load capacitors (8 MHz)
C4	100 nF	Ceramic	1	RDA5807M VCC decoupling
C5	1 µF	Ceramic	1	PAM8403 input coupling (mono-mixed AF_MONO → INL only)
C6	10 µF	Electrolytic	1	Headphone L coupling (LOUT to jack TIP)
C7	10 µF	Electrolytic	1	Headphone R coupling (ROUT to jack RING)
C8	100 pF	Ceramic	1	Antenna coupling (jack TIP to RDA ANT pin)
C9	10 µF	Electrolytic	1	HT7333 LDO input bypass
C10	10 µF	Electrolytic	1	HT7333 LDO output bypass
C11	10 µF	Ceramic	1	PAM8403 PVCC bypass
C12	10 µF	Ceramic	1	TP4056 input bypass (USB-C VBUS)
C13	100 nF	Ceramic	1	CH340C VCC decoupling
C14	100 nF	Ceramic	1	Auto-reset cap (CH340C DTR# → ATmega RESET)
C15	1 µF	Ceramic	1	TP4056 output bypass
C16	10 µF	Electrolytic	1	RDA5807M VCC bulk
C17	1 µF	Ceramic	1	TP4056 BAT pin decoupling
C18	100 nF	Ceramic	1	CH340C V3 pin (pin 4) bypass — required for USB enumeration
C19	100 nF	Ceramic	1	ADC filter: A0 divider midpoint to GND. Reduces source impedance for ATmega328P ADC (100 kΩ ‖ 100 kΩ = 50 kΩ too high without cap).
C20	10 µF	Ceramic	1	PAM8403 PVCC pin 15 bypass (each PVCC pin needs its own bypass cap)
C21	100 nF	Ceramic	1	ATmega AREF pin to GND — stabilises ADC reference voltage

4.4 Connectors, Switches, and Mechanical

Ref	Part	Qty	Notes
J1	3.5 mm TRS audio jack with detect pin	1	PJ-320A **NO-type** (Normally Open detect). When no plug is inserted, the detect pin floats; when a plug is inserted, the detect pin connects to sleeve (GND). This is the variant with a **5th pin** for detection. Do **not** use the NC (Normally Closed) variant.
J2	JST PH 2-pin connector	1	LiPo battery connector
J3	2×3 pin header (2.54 mm)	1	AVR ISP programming header — bootloader burn (one-time)
J4	USB-C receptacle (SMD, 16-pin)	1	**Both charging and programming.** Use GCT USB4105 or equivalent **16-pin** SMD receptacle. D+ / D- to CH340C; VBUS to TP4056. **Workshop solders.**
J5	1×4 pin header (2.54 mm)	1	OLED module connector (GND, VCC, SCL, SDA)
J6	1×5 pin header (2.54 mm)	2	Encoder connectors
J7	2×5 pin header (1.27 mm pitch)	1	RRD-102 radio module pads (castellated edge, 1.27 mm)
J8	1×2 pad or connector	1	Speaker output
J9	Solder pad	1	Backup wire antenna
—	Right encoder + knob cap	1	Tuning.
—	Left encoder + knob cap	1	Volume.
SW1	Latching push button	1	Power ON/OFF
LED1	Red 3 mm LED	1	Power indicator (from +3V3)
LED2	Red SMD LED (0805)	1	TP4056 CHG indicator (charging)
LED3	Green SMD LED (0805)	1	TP4056 STDBY indicator (full)
Y1	8 MHz crystal (HC49S)	1	ATmega clock source
SPK1	8 Ω 0.5–1 W speaker (28–40 mm)	1	Built-in speaker
BAT1	LiPo 3.7 V 800–1000 mAh flat	1	With JST PH connector

4.5 Mechanical (Enclosure)

Part	Qty	Notes
Clear acrylic sheet 2–3 mm	1	For laser cutting enclosure layers
M3×20 mm bolts	4	Corner bolts
M3 nuts	4	To hold bolts
M3×5 mm nylon standoffs	4	PCB mounting

Part	Qty	Notes
Rubber feet (adhesive)	4	Optional, for bottom

4.6 Estimated Cost

Item	Approx (Rs.)
ATmega328P-PU	300–500
PAM8403 Class-D amplifier (SOP-16)	80–150
CH340C USB-to-serial (SOP-16)	80–150
TP4056 charger IC (SOP-8)	30–60
HT7333 LDO (SOT-89)	30–60
DW01A + FS8205A battery protection	30–50
USB-C receptacle (SMD)	80–150
Crystal + caps + resistors + LEDs	150–300
3.5 mm audio jack	30–60
Small speaker (8 Ω)	100–200
LiPo 800–1000 mAh	400–800
2× encoders + knob caps	200–400
Latching push button	30–50
PCB fabrication (2-layer, 5 pcs, ∼70×50 mm)	800–1500
Acrylic cutting	200–500
M3 hardware	100–200
Total	**Rs. 2650–5130**

5 Complete Circuit Design

DANGER

1. **RDA5807M is a 3.3 V device.** Never connect 5 V to VCC. Instant destruction.
2. **I2C pull-ups go to +3V3 only.** Never to 5 V or VBAT.
3. **LiPo battery safety:** Never short-circuit. Never charge unattended. Never puncture.
4. **TP4056 PROG resistor (R9):** Sets charge current ($I = 1000/R9$). 2.2 kΩ = 455 mA, 10 kΩ = 100 mA, 1.2 kΩ = 1 A. Match to battery capacity.
5. **PAM8403 outputs are BTL:** Neither speaker wire goes to GND. Bridging either to GND will damage the IC.
6. **USB-C data lines:** D+ and D− go to CH340C only. Do not short VBUS to D+/D−.
7. **Electrolytic capacitor polarity matters.** Reversed = burst/fire.

5.1 ATmega328P Pin Assignment

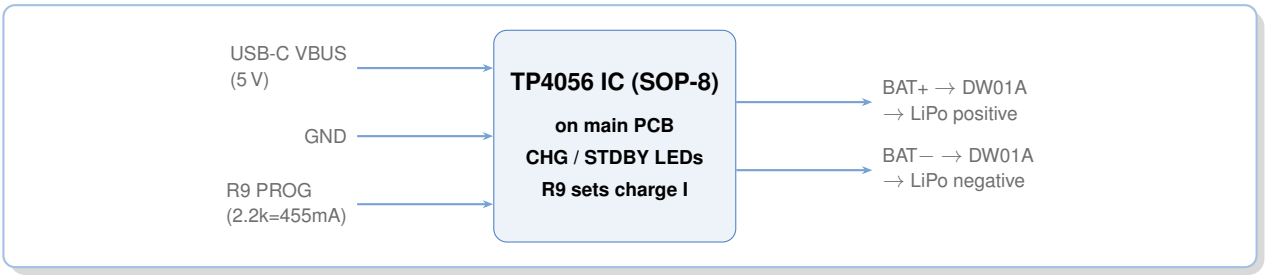
ATmega Pin	Arduino Pin	Net Name	Function
1 (PC6)	RESET	RESET	10 kΩ pull-up to +3V3 + ISP pin 5 + CH340C DTR# via C14 (100 nF auto-reset)
2 (PD0)	D0 (RX)	UART_RX	CH340C TXD — receives data from USB-C
3 (PD1)	D1 (TX)	UART_TX	CH340C RXD — transmits data to USB-C
4 (PD2)	D2 (INT0)	ENC_R_CLK	Right encoder CLK — hardware interrupt
5 (PD3)	D3 (INT1)	ENC_L_CLK	Left encoder CLK — hardware interrupt
6 (PD4)	D4	ENC_R_DT	Right encoder DT
7 (VCC)	VCC	+3V3	Power + C1 (100 nF) to GND
8 (GND)	GND	GND	Digital ground
9 (PB6)	XTAL1	XTAL1	8 MHz crystal + C3a (22 pF) to GND
10 (PB7)	XTAL2	XTAL2	8 MHz crystal + C3b (22 pF) to GND
11 (PD5)	D5	ENC_L_DT	Left encoder DT
12 (PD6)	D6	ENC_R_SW	Right encoder button (press = Seek UP)
13 (PD7)	D7	ENC_L_SW	Left encoder button (press = Seek DOWN)
14 (PB0)	D8	HP_DETECT	3.5 mm jack detect (LOW = plugged in)
15 (PB1)	D9	AMP_EN	PAM8403 ~SHDN pin. HIGH = amplifier on, LOW = mute. MCU-controlled speaker muting when HP detected on D8.
16 (PB2)	D10	—	Unused (available)
17 (PB3)	D11 (MOSI)	ISP_MOSI	ISP header pin 4
18 (PB4)	D12 (MISO)	ISP_MISO	ISP header pin 1
19 (PB5)	D13 (SCK)	ISP_SCK	ISP header pin 3
20 (AVCC)	AVCC	+3V3	Analog power + C2 (100 nF) to GND
21 (AREF)	AREF	—	C21 (100 nF) to GND for ADC reference stability. Leave electrically floating (internal AREF).
22 (GND)	AGND	GND	Analog ground
23 (PC0)	A0	BAT_SENSE	Voltage divider midpoint (VBAT/2)
24 (PC1)	A1	—	Unused
25 (PC2)	A2	—	Unused
26 (PC3)	A3	—	Unused

ATmega Pin	Arduino Pin	Net Name	Function
27 (PC4)	A4 (SDA)	I2C_SDA	I2C data: RDA5807M + SSD1306
28 (PC5)	A5 (SCL)	I2C_SCL	I2C clock: RDA5807M + SSD1306

Total GPIOs used: 13 of 23 available. Plenty of headroom for future features.

5.2 TP4056 LiPo Charger IC (SOP-8, On-Board)

The TP4056 is a **surface-mount CC/CV LiPo charger IC** (SOP-8) soldered directly onto the main PCB by the university workshop. It shares the USB-C VBUS line for 5 V input. Battery protection is provided by a separate DW01A + FS8205A pair.



NOTE

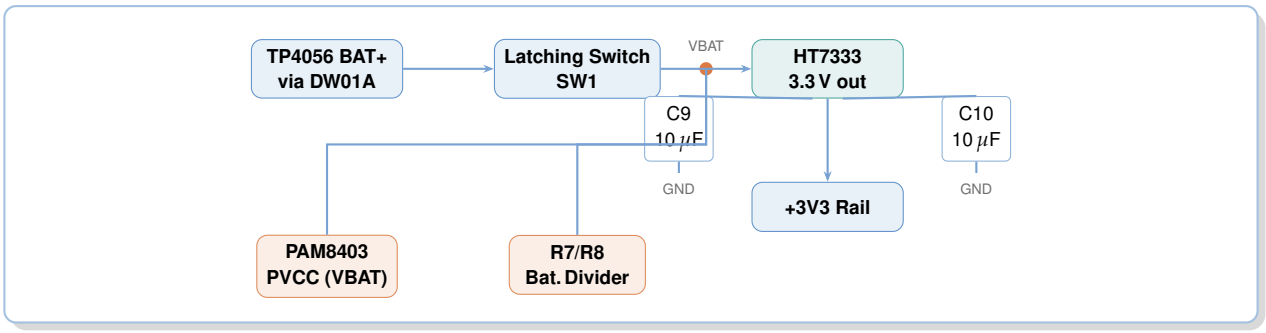
TP4056 IC on-board connections:

- **VCC (pin 4)** ← USB-C VBUS (5 V) + C12 (10 μF) bypass
- **PROG (pin 2)** → R9 to GND. 2.2 kΩ = ∼455 mA charge current (safe for 800 mAh cell, charges in ∼2 hrs). Use 10 kΩ for 100 mA (slow/safe), 1.2 kΩ for ∼1 A.
- **BAT (pin 3)** → DW01A/FS8205A → LiPo battery via JST J2. Plus C17 (1 μF) bypass.
- **CHRG (pin 7)** ← open-drain sink. Wire: USB VBUS → R12 (1 kΩ) → LED2 (red, anode to R12) → CHRG pin 7. The IC sinks current through the LED when charging.
- **STDBY (pin 6)** ← open-drain sink. Wire: USB VBUS → R13 (1 kΩ) → LED3 (green, anode to R13) → STDBY pin 6. The IC sinks current through the LED when battery is full.
- **GND (pin 1, 5)** → GND plane.
- **CE (pin 8)** → VCC (always enabled).

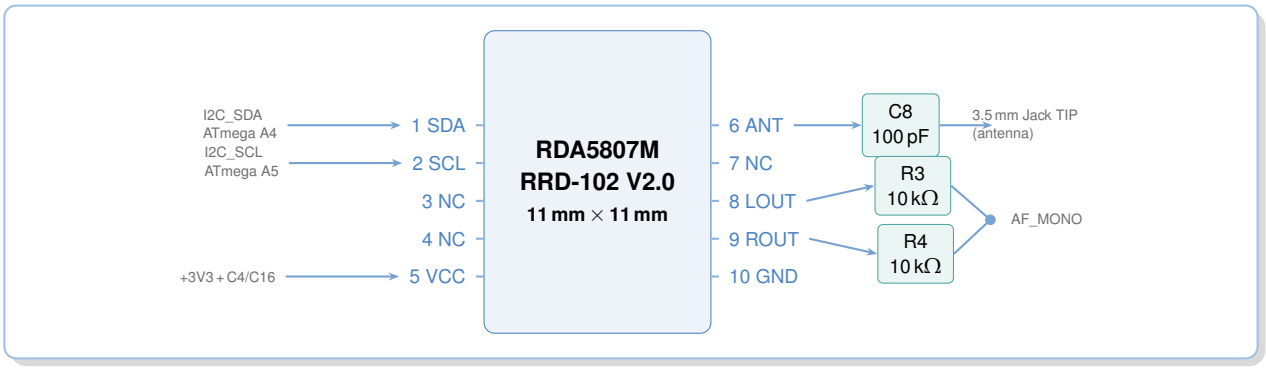
WARNING

Never place the power switch between the battery and BAT+! The DW01A protection circuit needs a permanent battery connection to monitor cell voltage. The latching power switch must be on the **VBAT line** (after DW01A output), not between battery and TP4056.

5.3 Power Switch and LDO

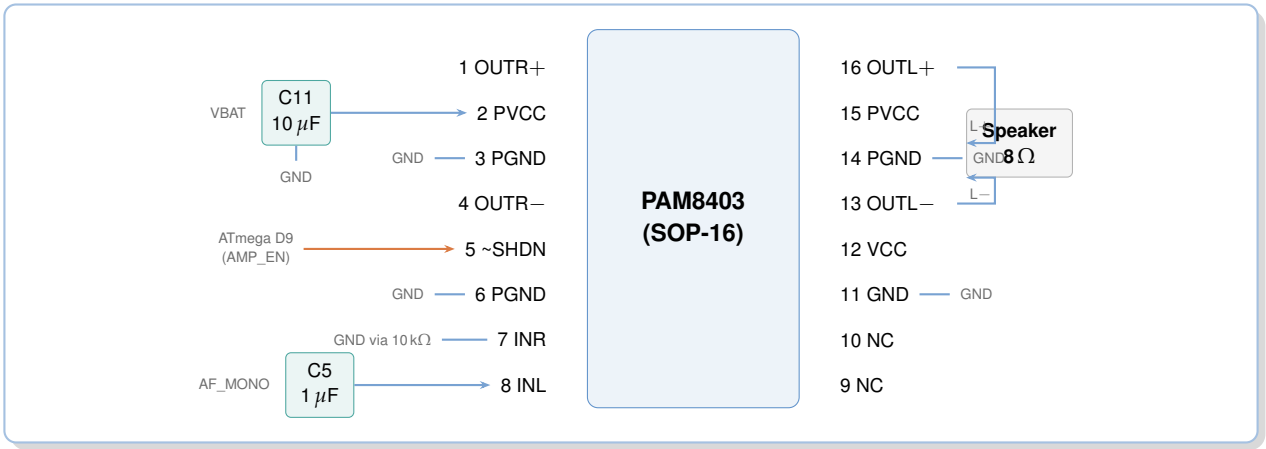


5.4 RDA5807M Module Connections



5.5 PAM8403 Class-D Amplifier (SOP-16, Stereo BTL)

The PAM8403 is a **surface-mount Class-D stereo amplifier** (SOP-16). In this design it is wired in **mono mode**: only the left input channel (INL, pin 8) receives the mono-mixed audio from R3/R4, and only the left BTL output (OUTL+/OUTL−) drives the 8 Ω speaker. The right channel input (INR, pin 7) is tied to GND via a 10 kΩ resistor to prevent noise. At VBAT 3.7 V into 8 Ω, expect ∼0.5 W output. The ~SHDN pin provides **MCU-controlled mute**: ATmega D9 drives it HIGH (amplifier on) or LOW (mute when headphones detected). **SMD — workshop solders.**



NOTE

PAM8403 connections (stereo BTL):

- **PVCC (pins 2, 15)** ← VBAT direct + C11 (10 µF ceramic) on pin 2 + C20 (10 µF ceramic) on pin 15. Each PVCC pin requires its own local bypass cap.
- **VCC (pin 12)** ← VBAT (logic supply)
- **PGND (pins 3, 6, 14)** and **GND (pin 11)** → GND plane (wide traces, short path)
- **INL (pin 8)** ← C5 (1 µF) from mono-mixed AF_MONO (R3+R4 sum of LOUT+ROUT)

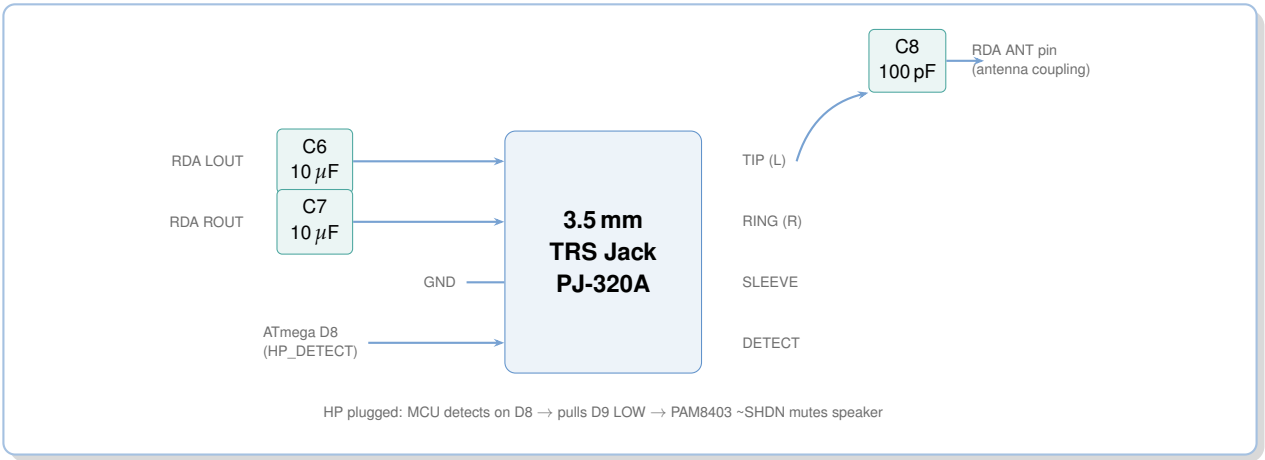
- **INR (pin 7)** → tied to GND via 10 kΩ (unused; prevents noise pickup)
- **OUTL+/OUTL− (pins 16, 13)** → Speaker (BTL — neither wire to GND)
- **OUTR+/OUTR− (pins 1, 4)** → unused (no connection, or leave NC)
- **~SHDN (pin 5)** ← ATmega D9. HIGH = amplifier on. LOW = shutdown (mute).

MCU-controlled speaker mute: When headphones are detected on D8, the MCU pulls D9 LOW to assert ~SHDN, shutting down the PAM8403. When headphones are removed, D9 goes HIGH and the speaker resumes.

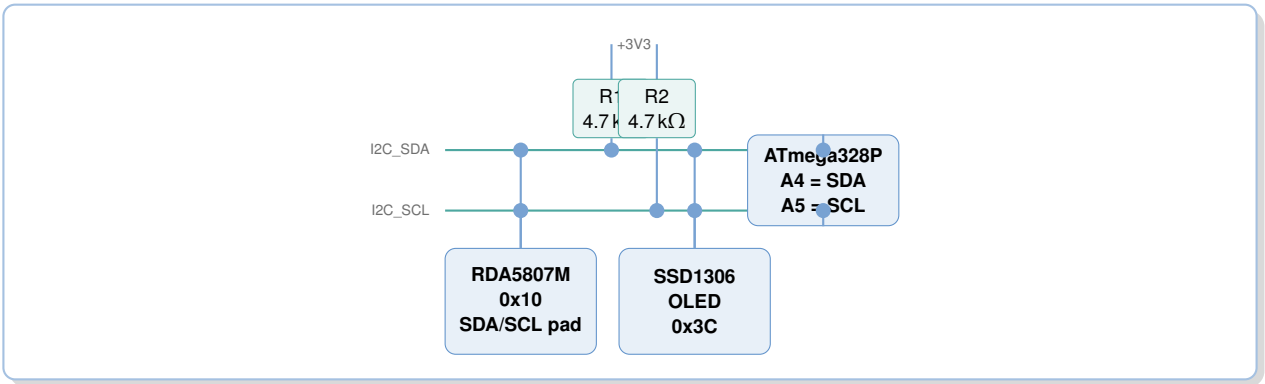
WARNING

PAM8403 outputs are BTL (Bridge-Tied Load). **Neither speaker wire goes to GND.** Connecting either speaker wire to ground will damage the IC. The speaker floats between OUTL+ and OUTL−.

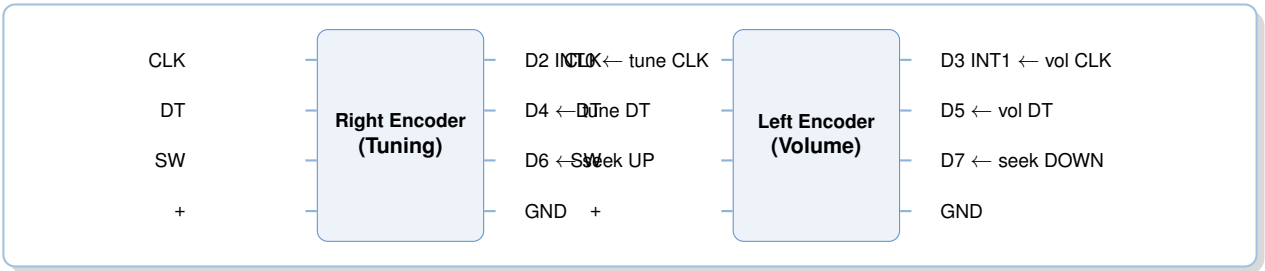
5.6 3.5 mm Headphone Jack



5.7 I2C Bus



5.8 Rotary Encoders

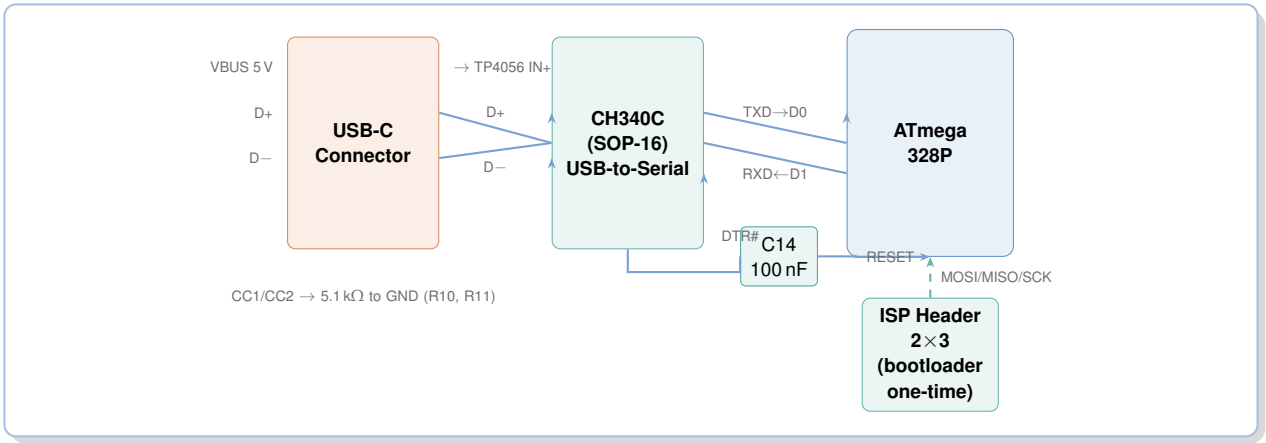


5.9 Programming Interface (CH340C + USB-C)

The CH340C is a **surface-mount USB-to-serial bridge** (SOP-16) permanently soldered on the PCB by the university workshop. It connects the USB-C data lines (D+/D−) to the ATmega UART (D0/D1), allowing firmware upload directly via the same USB-C port used for charging. No external crystal is needed — the CH340C has an internal oscillator.

WARNING

CH340C V3 pin (pin 4): This pin **must** be connected to +3V3 via a 100 nF bypass capacitor (C18) to GND. The V3 pin is the internal regulator output and **must not be left floating**—the chip will not enumerate on USB without it. At 3.3 V VCC, tie V3 directly to VCC and add C18 (100 nF) from V3 to GND.



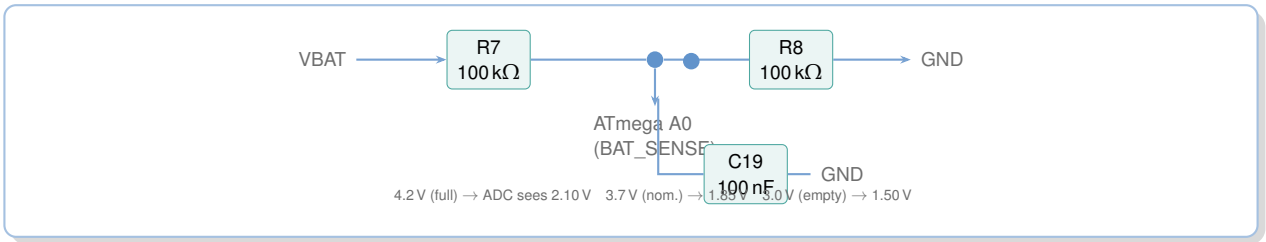
NOTE

Two-step programming workflow:

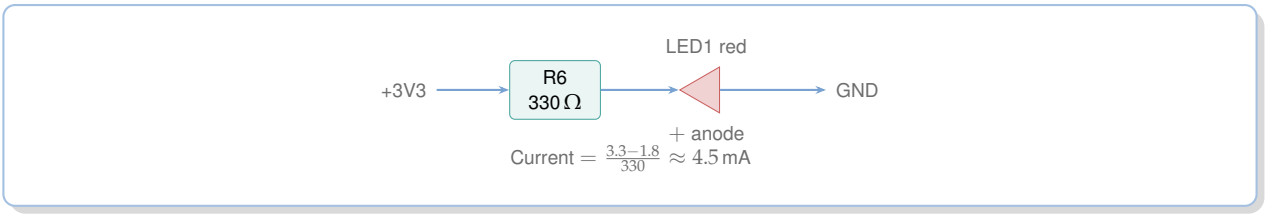
1. **One time only (bootloader):** Use an Arduino UNO as ISP (ArduinoISP sketch) connected to the 2×3 ISP header (J3). This burns the Arduino Pro Mini 3.3 V 8 MHz bootloader.
2. **Every firmware upload:** Plug a USB-C cable from your PC into J4. In Arduino IDE select "Arduino Pro Mini" → "3.3 V, 8 MHz" and the CH340 COM port. Upload as normal. Auto-reset via CH340C DTR# + C14 works the same as on any Arduino Pro Mini.

CH340C driver: Windows 10/11 usually auto-installs. If not, download the CH340 driver from WCH's website (free, one-time install).

5.10 Battery Voltage Divider



5.11 Power LED



5.12 Complete Connection Checklist

Use this as a verification checklist. Every wire in the project:

#	From	To	Check
1	ATmega pin 1 (RESET)	R5 (10 kΩ) to +3V3, ISP pin 5, C14 (100 nF) to CH340C DTR#	☐
2	ATmega pin 2 (D0/RX)	CH340C TXD	☐
3	ATmega pin 3 (D1/TX)	CH340C RXD	☐
4	ATmega pin 4 (D2)	Right encoder CLK	☐
5	ATmega pin 5 (D3)	Left encoder CLK	☐
6	ATmega pin 6 (D4)	Right encoder DT	☐
7	ATmega pin 7 (VCC)	+3V3 + C1(100nF) to GND	☐
8	ATmega pin 8 (GND)	GND bus	☐
9	ATmega pin 9 (XTAL1)	Y1 leg 1 + C3a(22pF) to GND	☐
10	ATmega pin 10 (XTAL2)	Y1 leg 2 + C3b(22pF) to GND	☐
11	ATmega pin 11 (D5)	Left encoder DT	☐
12	ATmega pin 12 (D6)	Right encoder SW	☐
13	ATmega pin 13 (D7)	Left encoder SW	☐
14	ATmega pin 14 (D8)	3.5mm jack DETECT pin (HP_DET)	☐
15	ATmega pin 15 (D9)	PAM8403 ~SHDN (AMP_EN: HIGH=on, LOW=mute)	☐
16	ATmega pin 17 (D11)	ISP header pin 4 (MOSI)	☐
17	ATmega pin 18 (D12)	ISP header pin 1 (MISO)	☐
18	ATmega pin 19 (D13)	ISP header pin 3 (SCK)	☐
19	ATmega pin 20 (AVCC)	+3V3 + C2(100nF) to GND	☐
20	ATmega pin 22 (AGND)	GND bus	☐
20b	ATmega pin 21 (AREF)	C21 (100 nF) to GND (ADC reference filter)	☐
21	ATmega pin 23 (A0)	R7/R8 divider midpoint + C19 (100 nF) to GND	☐
22	ATmega pin 27 (A4/SDA)	I2C_SDA bus	☐
23	ATmega pin 28 (A5/SCL)	I2C_SCL bus	☐
24	R1(4.7k)	I2C_SDA to +3V3	☐
25	R2(4.7k)	I2C_SCL to +3V3	☐
26	RDA5807M pad 1 (SDA)	I2C_SDA bus	☐

#	From	To	Check
27	RDA5807M pad 2 (SCL)	I2C_SCL bus	☐
28	RDA5807M pad 5 (VCC)	+3V3 + C4(100nF) + C16(10uF)	☐
29	RDA5807M pad 6 (ANT)	C8(100pF) to jack TIP + J9 pad	☐
30	RDA5807M pad 8 (LOUT)	R3(10k) to AF_MONO mix point; also C6(10μF) to jack TIP (stereo HP)	☐
31	RDA5807M pad 9 (ROUT)	R4(10k) to AF_MONO mix point; also C7(10μF) to jack RING (stereo HP)	☐
32	RDA5807M pad 10 (GND)	GND bus	☐
33	SSD1306 GND	GND bus	☐
34	SSD1306 VCC	+3V3	☐
35	SSD1306 SCL	I2C_SCL bus	☐
36	SSD1306 SDA	I2C_SDA bus	☐
37	PAM8403 INL (pin 8)	C5 (1 μF) from AF_MONO mix point (R3+R4 junction)	☐
38	PAM8403 INR (pin 7)	Tied to GND via 10 kΩ (unused; mono speaker only)	☐
39	PAM8403 PVCC (pins 2, 15)	VBAT + C11 (10 μF) on pin 2 + C20 (10 μF) on pin 15 — each PVCC needs its own bypass	☐
40	PAM8403 VCC (pin 12)	VBAT (logic supply)	☐
41	PAM8403 PGND (pins 3, 6, 14)	GND plane (wide trace, multiple vias)	☐
42	PAM8403 GND (pin 11)	GND plane	☐
43	PAM8403 OUTL+ (pin 16)	Speaker+ (SPK+ terminal J8)	☐
44	PAM8403 OUTL− (pin 13)	Speaker− (SPK− terminal J8)	☐
45	PAM8403 ~SHDN (pin 5)	ATmega D9 (AMP_EN)	☐
46	3.5 mm jack TIP	C6 from LOUT + C8 to RDA ANT	☐
47	3.5 mm jack RING	C7 from ROUT	☐
48	3.5 mm jack SLEEVE	GND	☐
49	3.5 mm jack DETECT	ATmega D8 (HP_DET)	☐
50	USB-C VBUS	TP4056 VCC (pin 4) + C12 bypass	☐
51	USB-C D+	CH340C UD+	☐
52	USB-C D−	CH340C UD−	☐
53	USB-C CC1	R10 (5.1 kΩ) to GND	☐
54	USB-C CC2	R11 (5.1 kΩ) to GND	☐
55	USB-C GND	GND bus	☐
56	CH340C TXD	ATmega pin 2 (D0/RX)	☐

#	From	To	Check
57	CH340C RXD	ATmega pin 3 (D1/TX)	☐
58	CH340C DTR#	C14 (100 nF) to ATmega RESET (auto-reset)	☐
59	CH340C VCC	+3V3 + C13 (100 nF) bypass	☐
59b	CH340C V3 (pin 4)	+3V3 + C18 (100 nF) bypass to GND — required for USB enumeration	☐
60	TP4056 BAT (pin 3)	DW01A → LiPo J2 positive + C17 (1μF)	☐
61	TP4056 PROG (pin 2)	R9 (2.2 kΩ) to GND (∼455 mA charge current)	☐
62	TP4056 CHRG (pin 7)	VBUS → R12 (1 kΩ) → LED2 (red anode) → CHRG pin 7 (open-drain sink)	☐
63	TP4056 STDBY (pin 6)	VBUS → R13 (1 kΩ) → LED3 (green anode) → STDBY pin 6 (open-drain sink)	☐
64	DW01A/FS8205A	Low-side on BAT− path: LiPo BAT− → FS8205A drain; FS8205A source → R14 (100Ω) → GND. DW01A CS pin to midpoint. Protects against over-discharge/overcharge/short-circuit.	☐
65	TP4056 BAT+ (via latching SW1)	SW1 input → VBAT bus. DW01A/FS8205A is on BAT− (low-side), NOT on BAT+.	☐
66	SW1 output	VBAT bus	☐
67	VBAT	HT7333 IN + PAM8403 PVCC + R7 (100 kΩ)	☐
68	HT7333 OUT	+3V3 bus + C9 (in) + C10 (out) bypasses	☐
69	Power LED	+3V3 — R6 (330 Ω) — LED1 — GND	☐

6 KiCad 9.0 — Schematic Entry (Step by Step)

6.1 Opening the Project

1. Open KiCad 9.0
2. File → Open Project → navigate to `ESP32_FM_Radio_V2/`
3. Double-click the `.kicad_sch` file to open the schematic editor
4. The schematic is blank — we start from scratch

NOTE

Optional: Rename the project folder to something like `FM_Pocket_Radio_V3/` to avoid confusion with the old ESP32 project name.

6.2 Setting Up Schematic Sheets

For a clean layout, use one sheet with logical grouping:

- Top-left: Power supply (USB-C + TP4056 + DW01A + HT7333 + switch)
- Top-right: MCU (ATmega328P + crystal + CH340C + ISP header)
- Bottom-left: Radio (RDA5807M + I2C + antenna)
- Bottom-right: Audio (PAM8403 + speaker + headphone jack)
- Center: Display + encoders

6.3 Placing Components

Press **A** (Add Symbol) to place components. Search by name:

Component	KiCad Library	Symbol Name
ATmega328P-PU	MCU_Microchip_ATmega	ATmega328P-PU
ATmega328P-AU (alt.)	MCU_Microchip_ATmega	ATmega328P-AU
Crystal 8 MHz	Device	Crystal
Resistor	Device	R
Capacitor	Device	C
Capacitor polarized	Device	C_Polarized
LED	Device	LED
Power flag	power	PWR_FLAG
+3V3	power	+3V3
GND	power	GND
Custom symbols (must import — see Section 7):		
RDA5807M module	Custom lib	RDA5807M_Module
PAM8403	Custom lib	PAM8403
TP4056 IC (SOP-8)	Custom lib	TP4056
CH340C	Custom lib	CH340C
HT7333 (SOT-89)	Custom lib	HT7333
DW01A + FS8205A	Custom lib	DW01A / FS8205A
USB-C connector	Connector_USB	USB_C_Receptacle_USB2.0
SSD1306 OLED	Custom lib	SSD1306_Module
3.5 mm Audio Jack	Connector_Audio	AudioJack3_SwitchT
Rotary Encoder	Device	RotaryEncoder_Switch

6.4 Wiring

1. Press **W** to start a wire
2. Connect pins as per the connection table in Section 5

3. Use **labels** (press **L**) for long-distance connections: `I2C_SDA`, `I2C_SCL`, `+3V3`, `VBAT`, `AF_MONO`, etc.
4. Add **power flags**: place `PWR_FLAG` symbols on +3V3 and GND to avoid ERC warnings
5. Add net labels on all signal wires for clarity

6.5 Annotation and Values

1. Edit → Annotate Schematic (or press **Ctrl+Shift+A**)
2. Assigns R1, R2, C1, C2, U1, etc. automatically
3. Double-click each component to set its **Value**: "4.7k", "100nF", "ATmega328P-PU", etc.

6.6 Running ERC

1. Inspect → Electrical Rules Checker (or toolbar icon)
2. Click "Run ERC"
3. Fix all errors. Common issues:
   - Missing power flags on power rails — add PWR_FLAG symbols
   - Unconnected pins — add "no connect" flags (press **Q**)
   - Pin type conflicts — verify power pin directions
4. **Zero errors required** before proceeding to PCB layout

7 Importing Custom Components in KiCad 9.0

Several components are NOT in KiCad's default libraries. You must import or create them.

7.1 Method 1: Download from SnapEDA (Recommended)

1. Go to `https://www.snapeda.com`
2. Search for the component (e.g., "PAM8403", "CH340C", "RDA5807M")
3. Click "Download Symbol & Footprint"
4. Select "KiCad" format
5. Download the ZIP file
6. Extract — you get `.kicad_sym` (symbol) and `.kicad_mod` (footprint) files
7. **Place files:** Create a folder `libs/` inside your KiCad project directory. Copy the downloaded files there.

7.1.1 Register Symbol Library

1. In KiCad Schematic Editor: Preferences → Manage Symbol Libraries
2. Click the **Project Specific Libraries** tab
3. Click the "+" (add) button
4. For "Library Path", browse to your `libs/something.kicad_sym` file
5. Give it a nickname (e.g., "PAM8403")
6. Click OK

7.1.2 Register Footprint Library

1. In KiCad PCB Editor: Preferences → Manage Footprint Libraries
2. Click the **Project Specific Libraries** tab
3. Click "+" button
4. For "Library Path", browse to the `libs/` folder containing the `.kicad_mod` file
5. Give it a nickname
6. Click OK

7.2 Method 2: Download from Ultra Librarian

1. Go to `https://www.ultralibrarian.com`
2. Search for the component
3. Click "Download" → select "KiCad" format
4. Follow the same import steps as SnapEDA above

7.3 Method 3: Create Custom Symbols Manually

For simple modules (OLED, RDA5807M breakout), it's easier to create custom symbols:

7.3.1 Creating a Symbol for the RDA5807M Module (10-pin)

1. In KiCad: Tools → Symbol Editor
2. File → New Library → save as `libs/custom.kicad_sym` (Project type)
3. Click "New Symbol" icon
4. Name: `RRD102_RDA5807M`
5. Draw a rectangle body
6. Add 10 pins:
   - Left side (input): Pin 1 SDA, Pin 2 SCL, Pin 3 NC, Pin 4 NC, Pin 5 VCC
   - Right side (output): Pin 6 ANT, Pin 7 NC, Pin 8 LOUT, Pin 9 ROUT, Pin 10 GND
7. Set pin types: SDA/SCL = Bidirectional, VCC = Power Input, GND = Power Input, LOUT/ROUT = Output, ANT = Passive, NC = Unconnected
8. Save. The symbol is now available in your schematic.

7.3.2 Creating a Footprint for the RDA5807M Module

1. In KiCad: Tools → Footprint Editor
2. File → New Library → save in `libs/` folder
3. New Footprint → name: `RRD102_RDA5807M`
4. The module is 11 mm × 11 mm with two rows of 5 pads at 1.27 mm pitch
5. Add module pads (castellated edge or through-hole header as appropriate):
   - Left column (pads 1–5): X = -4.5 mm, Y starts at -2.54 mm, step 1.27 mm
   - Right column (pads 6–10): X = +4.5 mm, same Y positions
   - Pad size: 0.8 mm × 1.5 mm (rectangular) for castellated; or use through-hole drill 0.8 mm
6. Draw courtyard (fabrication layer): 11 mm × 11 mm rectangle centered

7. Save

7.3.3 Creating a Symbol/Footprint for the SSD1306 OLED Module

This is simple — it's just a 4-pin header:

- Symbol: 4 pins (GND, VCC, SCL, SDA) in a DIP-like shape
- Footprint: use KiCad's built-in `Connector_PinHeader_1x04_P2.54mm_Vertical`

8 KiCad 9.0 — PCB Layout (Step by Step)

8.1 Transfer Schematic to PCB

1. In Schematic Editor: Tools → Update PCB from Schematic (or **F8**)
2. Click "Update PCB"
3. All components appear as a "rat's nest" in the PCB editor

8.2 Assign Footprints

Before updating PCB, each schematic symbol needs a footprint. In Schematic Editor:

1. Tools → Edit Symbol Fields (bulk edit)
2. For each component, set the Footprint column:

Component	Footprint
ATmega328P-PU	Package_DIP:DIP-28_W7.62mm
PAM8403 (SOP-16)	Package_SO:SOP-16_4.4x10.4mm_P1.27mm
CH340C (SOP-16)	Package_SO:SOP-16_4.4x10.4mm_P1.27mm
TP4056 (SOP-8)	Package_SO:SOIC-8_3.9x4.9mm_P1.27mm
HT7333 (SOT-89)	Package_TO_SOT_SMD:SOT-89-3
DW01A (SOT-23)	Package_TO_SOT_SMD:SOT-23
FS8205A (SOT-23-6)	Package_TO_SOT_SMD:SOT-23-6
USB-C receptacle	Connector_USB:USB_C_Receptacle_GCT_USB4105
Crystal 8 MHz	Crystal:Crystal_HC49-4H_Vertical
Resistors (TH)	Resistor_THT:R_Axial_DIN0207_L6.3mm_D2.5mm_P10.16mm
Caps ceramic (TH)	Capacitor_THT:C_Disc_D5.0mm_W2.5mm_P2.50mm
Caps electrolytic	Capacitor_THT:CP_Radial_D5.0mm_P2.50mm
LEDs TH (3mm)	LED_THT:LED_D3.0mm
LEDs SMD (0805)	LED_SMD:LED_0805_2012Metric
3.5 mm Jack + SW	Connector_Audio:Jack_3.5mm_PJ320A
JST battery	Connector_JST:JST_PH_B2B-PH-K_1x02_P2.00mm_Vertical
ISP header (2×3)	Connector_PinHeader_2x03_P2.54mm_Vertical
OLED header (1×4)	Connector_PinHeader_1x04_P2.54mm_Vertical
Encoder headers (1×5)	Connector_PinHeader_1x05_P2.54mm_Vertical
RRD-102 module pads	Custom footprint (11×11mm, 2×5, 1.27mm pitch)

Component	Footprint
Speaker pads	Connector_JST:JST_PH_B2B-PH-K_1x02_P2.00mm_Vertical

NOTE

This design uses a **mix of SMD and through-hole** on a **2-layer PCB**. The university workshop solders all SMD components (PAM8403, CH340C, TP4056, HT7333, DW01A, FS8205A, USB-C). You solder through-hole parts and plug in modules at home.

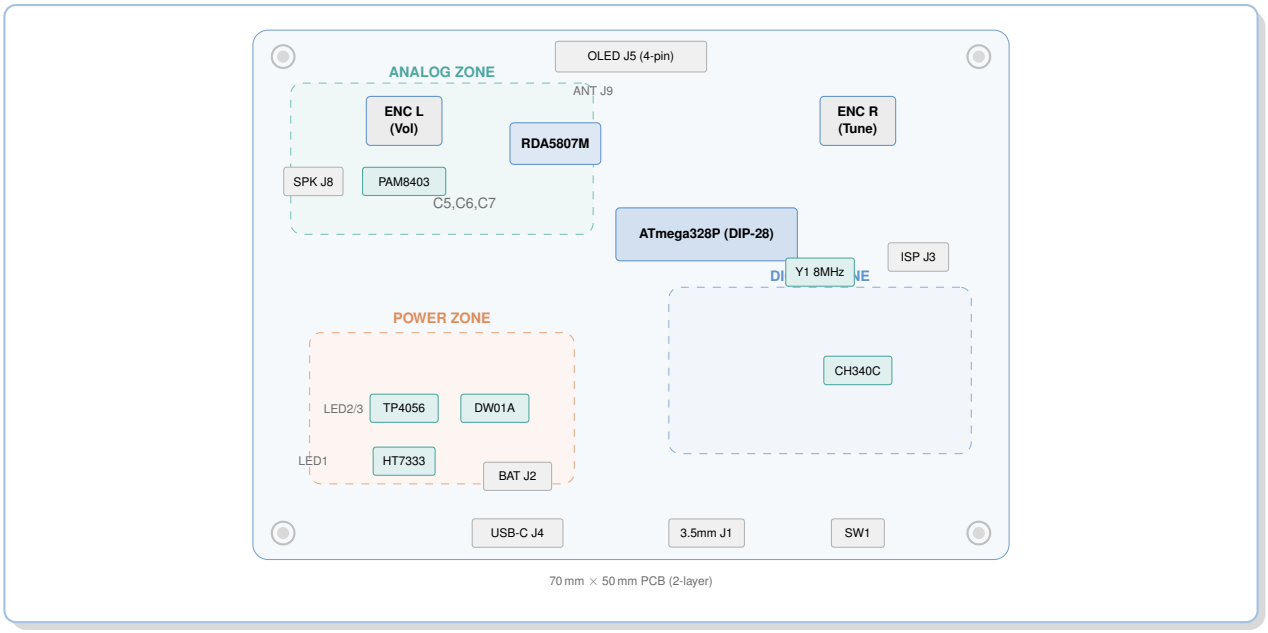
8.3 Set Board Outline

1. Switch to **Edge.Cuts** layer
2. Draw a rectangle: 70 mm × 50 mm (or adjust to fit)
3. Round corners: select edges → right-click → fillet → 2 mm radius
4. Add 4× M3 mounting holes at corners (3.2 mm drill, 6 mm annular ring)
   - Place → Footprint → search "MountingHole_3.2mm_M3"
   - Position at (3.5, 3.5), (66.5, 3.5), (3.5, 46.5), (66.5, 46.5)

8.4 Component Placement Guidelines

1. **South edge (user-facing):** USB-C (J4), 3.5 mm jack (J1), latching power button (SW1) — all on the same side for ergonomic access.
2. **North edge:** OLED module header (J5) — display faces up through acrylic window.
3. **Top surface:** Right encoder (tuning, east side) and left encoder (volume, west side) near north edge for thumb access.
4. **Centre:** ATmega328P (DIP-28) in the centre of the board.
5. **Analog zone (west half):** PAM8403, speaker output (J8), 3.5 mm jack, headphone coupling caps, RDA5807M module. Keep audio traces short and away from digital signals.
6. **Digital zone (east half):** CH340C, USB-C, MCU digital pins, ISP header, crystal.
7. **Power zone (south-west corner):** TP4056, DW01A/FS8205A, HT7333, battery JST (J2), power switch.
8. **Radio module (north-east or north-centre):** RDA5807M away from amplifier and MCU noise. Antenna pad (J9) nearby.
9. **Decoupling caps:** within 2 mm of their respective IC power pins.
10. **Crystal:** close to ATmega XTAL1/XTAL2 pins (short traces, surrounded by ground).
11. **Test points:** Add exposed copper pads (TP style) for +3V3, VBAT, GND, and I2C_SDA/SCL for easy multimeter/scope probing during debugging.
12. **Ground plane:** continuous B.Cu ground plane. SMD pads on F.Cu with vias to B.Cu ground.

8.5 PCB Component Layout Diagram (Top View)



8.6 Noise Reduction and Signal Integrity

This 2-layer PCB uses a continuous back-copper ground plane and strategic zone separation to minimise noise coupling between the digital MCU, RF radio, and Class-D amplifier.

1. **Ground plane (B.Cu):** Fill the entire back layer with GND copper. This provides a low-impedance return path for all signals and shields the digital traces from the analog audio.
2. **Analog/digital separation:** Route analog audio traces (RDA5807M LOUT/ROUT, PAM8403 inputs/outputs, headphone coupling) on the west half of F.Cu. Route digital traces (SPI, UART, encoder signals) on the east half. The ATmega sits at the boundary.
3. **PAM8403 ground returns:** Use wide (0.5 mm+) traces from PAM8403 PGND pins to the nearest ground via. Multiple vias around the PAM8403 reduce ground inductance. The Class-D switching currents must return to ground as locally as possible.
4. **R3/R4 mixing resistors:** Place these immediately adjacent to the PAM8403 input pins. Short audio traces reduce noise pickup.
5. **Crystal guard ring:** Place ground vias around the 8 MHz crystal and C3a/C3b. This prevents crystal harmonics from coupling into the RDA5807M receiver.
6. **TP4056 / HT7333 input/output capacitors:** Place C9, C10, C12, C15, C17 within 2 mm of the IC power pins. Short traces between capacitor and pin reduce ESL.
7. **Antenna area:** No copper pour or digital traces within 3 mm of the RDA5807M antenna pad (J9) and the coupling capacitor C8. The antenna trace to the 3.5 mm jack should be as direct as possible.
8. **USB-C / CH340C:** Keep the USB D+/D− traces short and matched-length (under 20 mm). Place CH340C as close to the USB-C connector as practical.
9. **Star-point power distribution:** VBAT fans out from the switch to three destinations: HT7333 IN, PAM8403 PVCC, and R7 divider. Use separate traces (not daisy-chained) to prevent amplifier current spikes from coupling into the LDO.

WARNING

Keep the antenna area clean. No ground plane directly under the RDA5807M antenna pad. No high-speed digital traces near the antenna coupling path (C8 → jack). This minimizes RF interference.

8.7 Design Rules Setup

Edit → Board Setup → Design Rules:

Parameter	Value	Notes
Minimum trace width	0.25 mm	Default signal traces
Power trace width	0.5–1.0 mm	+3V3, VBAT, GND
Minimum clearance	0.2 mm	Between traces
Minimum via drill	0.3 mm	For signal vias (F.Cu ↔ B.Cu ground plane)
Via annular ring	0.6 mm outer diameter	
Copper to edge	0.3 mm	Keep copper away from board edge

8.8 Routing Traces

1. Press **X** to start routing a trace
2. **Two-layer strategy:** F.Cu for signal traces and component pads. B.Cu is a continuous GND pour. Use vias to connect ground pads to the B.Cu plane.
3. Route **power traces first**: VBAT, +3V3 — use 0.5–1.0 mm width on F.Cu
4. Route **I2C bus**: SDA and SCL — use 0.3 mm, short runs
5. Route **audio traces**: PAM8403 input/output — keep in analog zone, away from digital signals
6. Route **USB data**: CH340C D+/D− traces matched length, under 20 mm
7. Route **encoder signals**: short runs from edge connectors to MCU
8. Route all remaining signals

8.9 Ground Plane Pour

1. Select **B.Cu** (back copper) layer
2. Place → Add Fill Zone (or press **Ctrl+Shift+Z**)
3. Draw a rectangle covering the entire board
4. Assign net: GND
5. Press **B** to fill all zones
6. This creates a continuous ground plane on B.Cu, providing a low-impedance return path and shielding analog from digital. Through-hole GND legs and SMD GND pads connect via stitching vias.

NOTE

A solid B.Cu ground plane is essential for this mixed SMD/TH design. It provides a low-impedance return path for the Class-D amplifier switching currents, shields the FM radio receiver from MCU digital noise, and ensures the USB-C data lines have proper impedance control.

8.10 Silkscreen

1. Add text labels on F.Silkscreen layer:
   - Project name: "FM Pocket Radio V3"
   - Component references (auto-placed by KiCad)
   - Pin labels near headers: "SDA", "SCL", "TX", "RX", "MISO", etc.
   - Polarity markers near electrolytic caps and LEDs

8.11 Running DRC

1. Inspect → Design Rules Checker
2. Click "Run DRC"
3. Fix all errors:
   - Trace clearance violations — reroute offending traces
   - Unconnected nets — add missing traces or vias
   - Minimum width violations — widen undersized traces
4. **Zero errors required** before generating Gerber files

9 Generating Gerber Files

1. In PCB Editor: File → Fabrication Outputs → Gerbers (.gbr)
2. Set output directory: `gerbers/`
3. Select layers to export:
   - F.Cu (front copper — signal traces and SMD pads)
   - B.Cu (back copper — continuous GND ground plane)
   - F.Silkscreen (component labels, reference designators)
   - F.Mask (solder mask, top side)
   - B.Mask (solder mask, bottom side)
   - F.Paste (solder paste for SMD pads — needed for reflow/hand soldering)
   - Edge.Cuts (board outline)
4. Click "Plot" to generate all Gerber files
5. File → Fabrication Outputs → Drill Files (.drl)
   - Format: Excellon
   - Drill origin: Absolute
   - Click "Generate Drill File"
6. **Verify:** Open Gerber files in KiCad's Gerber Viewer (File → Open in new Gerber Viewer). Check each layer visually. Confirm all traces, pads, and board outline are correct.

9.1 BOM and Pick-and-Place Export (If Shop Does Assembly)

1. File → Fabrication Outputs → Component Placement (.pos)
   - Format: CSV
   - Include: all through-hole components
   - This file tells the shop where to place each component
2. In Schematic Editor: Tools → Edit Symbol Fields → Export as CSV
   - This generates the BOM (Bill of Materials)

9.2 Sending to PCB Shop

Create a ZIP file containing:

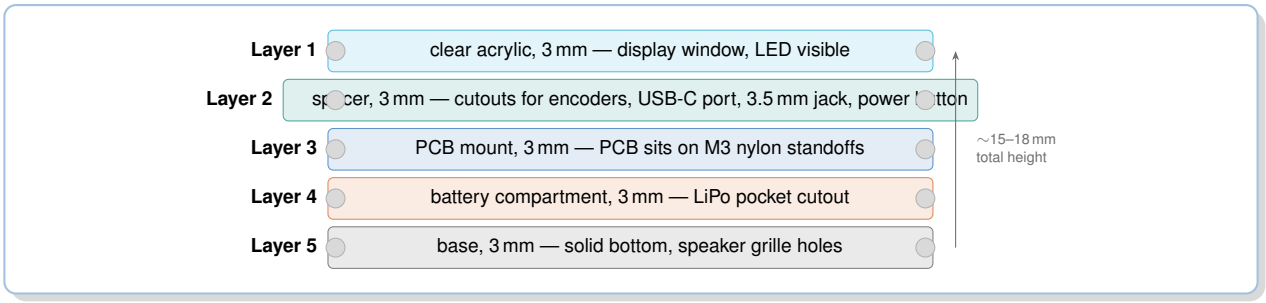
- All Gerber files (.gbr)
- Drill file (.drl)
- BOM (if assembly needed)
- Pick-and-place file (if assembly needed)
- A note specifying: **2-layer (double-sided)**, FR-4, 1.6 mm thickness, HASL finish, green solder mask; mixed SMD + through-hole. Workshop assembles SMD components (PAM8403, CH340C, TP4056, HT7333, DW01A, FS8205A, USB-C, SMD LEDs).

Upload or email to the shop (e.g., LankaPCB, Nilambara, etc.).

10 Enclosure Design — Laser-Cut Acrylic

10.1 Concept: Stacked Acrylic Layers

The enclosure is made by stacking 5–7 layers of laser-cut clear (or tinted) acrylic, held together with M3 bolts through the corners. Each layer has different cutouts.



10.2 Layer Drawings (Dimensions in mm)

All layers are 76 mm × 56 mm (PCB + 3 mm margin on each side).

Layer 1 — Top (clear acrylic, 3 mm):

- 4× M3 corner holes (3.2 mm drill)
- Display window area: no cutout needed (clear acrylic — OLED visible through it)
- Optional: frosted area over non-display zone for aesthetics

Layer 2 — Component Clearance Spacer (3 mm):

- 4× M3 corner holes
- Large rectangular cutout in center (for OLED, encoders, tall components)
- Side cutouts for: encoder shafts, USB-C port, 3.5 mm jack, power button

Layer 3 — PCB Mount (3 mm):

- 4× M3 corner holes (outer)
- 4× M3 holes matching PCB mount holes (for standoffs)
- Side cutouts for: USB-C port, 3.5 mm jack, power button

Layer 4 — Battery Space (3–6 mm, may use 2 layers):

- Large cutout for LiPo battery (∼50 mm × 35 mm × 5 mm)
- Side cutout for battery wires

Layer 5 — Bottom (3 mm, opaque or clear):

- 4× M3 corner holes
- Speaker grille: array of 2 mm holes in a circular pattern (∼30 mm diameter) centered under speaker
- Rubber feet mounting points

10.3 Design in Fusion 360

1. Export PCB as STEP: In KiCad PCB Editor, File → Export → STEP
2. Open Fusion 360
3. Insert → Insert Mesh/STEP → import the PCB STEP file
4. Create each acrylic layer as a sketch + extrude (3 mm each)
5. Add cutouts using the PCB model as reference for component positions
6. Export each layer as DXF: right-click face → Save as DXF
7. Send DXF files to laser cutting shop

NOTE

Alternative to Fusion 360: You can design the 2D layer profiles in **Inkscape** (free) or **LibreCAD** (free). Draw each layer as a 2D outline with cutouts, export as DXF or SVG. This is faster if you're not comfortable with Fusion 360.

10.4 Assembly

1. Stack layers bottom to top: Layer 5 → 4 → 3 → 2 → 1
2. Insert M3×20mm screws from bottom up through all 4 corner holes
3. Place nylon standoffs between Layer 3 and PCB
4. Mount PCB on standoffs
5. Tighten M3 nuts on top (or use acorn nuts for clean look)
6. Battery sits in Layer 4 cutout, connected to PCB via JST connector

11 Firmware

11.1 Arduino IDE Setup

1. Open Arduino IDE (1.8.x or 2.x)
2. Tools → Board → "**Arduino Pro or Pro Mini**"
3. Tools → Processor → "**ATmega328P (3.3V, 8 MHz)**"

4. Tools → Programmer → "Arduino as ISP" (for bootloader)

11.2 Required Libraries

Install via Sketch → Include Library → Manage Libraries:

- **U8g2** by oliver (for SSD1306 OLED display)
- **Wire** (built-in, for I2C)
- **EEPROM** (built-in, replaces ESP32 Preferences/NVS)

11.3 Complete Firmware Source Code

NOTE

This firmware was designed with UI/UX prototyped first as a Python/Pillow simulation (see project repo). The three "premium" UX behaviours translated from the simulation:

- **Station name scrolling** — long names pan left/right automatically
- **Volume slot-machine tween** — old number slides out, new number slides in
- **Tactile flash** — white screen for one frame on button press, mimics physical "clack"

```
/*
 * FM Pocket Radio - ATmega328P Firmware (Rev 2 - Premium UX)
 * Board: "Arduino Pro or Pro Mini"
 * Processor: "ATmega328P (3.3V, 8 MHz)"
 * Programming: USB-C (on-board CH340C) --- same port as charging
 *
 * UX features ported from Python/Pillow simulation:
 *   - Station name auto-scroll for long names
 *   - Volume tween (slot-machine reel animation)
 *   - Tactile flash on button press
 */

#include <Wire.h>
#include <U8g2lib.h>
#include <EEPROM.h>

// ==================== PIN DEFINITIONS ====================
#define PIN_ENC_R_CLK  2   // Right encoder CLK (INT0 hardware interrupt)
#define PIN_ENC_L_CLK  3   // Left encoder CLK  (INT1 hardware interrupt)
#define PIN_ENC_R_DT   4   // Right encoder DT
#define PIN_ENC_L_DT   5   // Left encoder DT
#define PIN_ENC_R_SW   6   // Right encoder switch -> Seek UP
#define PIN_ENC_L_SW   7   // Left encoder switch  -> Seek DOWN
#define PIN_HP_DET     8   // Headphone detect (LOW = plugged in)
#define PIN_AMP_EN     9   // PAM8403 ~SHDN: HIGH = amplifier on, LOW = mute
#define PIN_BAT_ADC    A0  // Battery voltage divider midpoint

// ==================== RDA5807M ====================
#define RDA_ADDR    0x10
#define FREQ_MIN_10 875
#define FREQ_MAX_10 1080

U8G2_SSD1306_128X64_NONAME_F_HW_I2C oled(U8G2_R0, U8X8_PIN_NONE);
static uint16_t reg[8];

// ==================== RUNTIME STATE ====================
volatile int8_t encRDelta = 0;
volatile int8_t encLDelta = 0;
uint16_t freq10     = 939;
uint8_t  volume     = 8;
int      sigLevel   = 0;
int      batPercent = 100;
```

```
bool     hpPlugged  = false;
uint32_t lastChange = 0;
bool     needSave   = false;
uint32_t lastBtnR   = 0, lastBtnL = 0;
uint32_t lastDisp   = 0, bootTime = 0;
bool     booting    = true;

#define EE_MAGIC_ADDR 0
#define EE_FREQ_ADDR  1
#define EE_VOL_ADDR   3
#define EE_MAGIC_VAL  0xAA

// ==================== UX STATE ====================
bool     flashPending = false;   // tactile flash: white 1 frame
uint8_t  oldVolume    = 8;
int8_t   volAnimY     = 11;      // 11=settled, 0-10=animating

enum ScrollState {
  SCR_IDLE, SCR_WAIT_START, SCR_FWD, SCR_PAUSE_END,
  SCR_BACK, SCR_PAUSE_START
};
ScrollState scrollState = SCR_WAIT_START;
int16_t     scrollX     = 0;
uint32_t    scrollTimer = 0;
uint16_t    prevFreq10  = 0;

// ==================== STATION DATABASE (PROGMEM) ====================
typedef struct { uint16_t freq10; char name[16]; } Station;
const Station PROGMEM stations[] = {
  { 876,"Lite FM"        },{ 883,"E FM"           },
  { 886,"Singha FM"      },{ 891,"Isira TNL FM"   },
  { 896,"City FM"        },{ 904,"Vettri FM"      },
  { 909,"Shaa FM"        },{ 914,"Alai FM"        },
  { 922,"FM Derana"      },{ 927,"Y FM"           },
  { 930,"Gold FM"        },{ 935,"Lakhanda"       },
  { 948,"Neth FM"        },{ 956,"Rhythm FM"      },
  { 961,"Hiru FM"        },{ 966,"Kiriula FM"     },
  { 969,"Kiss FM"        },{ 971,"Real Radio"     },
  { 974,"Radio Sri Lanka"},{ 982,"Siyatha FM"     },
  { 987,"Sun FM"         },{ 992,"TNL Rocks"      },
  { 995,"Red FM"         },{1000,"Shree FM"       },
  {1005,"Ran FM"         },{1008,"Yes FM"         },
  {1018,"Seth FM"        },{1021,"Tamil NS"       },
  {1026,"Vasantham"      },{1034,"Sooriyan FM"    },
  {1039,"Shakthi FM"     },{1044,"Rangiri SL"     },
  {1047,"Thendral"       },{1052,"Lakviru FM"     },
  {1060,"Lak FM"         },{1065,"Sirasa FM"      },
  {1070,"V FM"           },{1073,"Kandurata"      },
};
const uint8_t NUM_STATIONS = sizeof(stations)/sizeof(Station);

void getStationName(uint16_t f, char* buf, uint8_t len) {
  Station s;
  for (uint8_t i = 0; i < NUM_STATIONS; i++) {
    memcpy_P(&s, &stations[i], sizeof(Station));
    int d = (int)f - (int)s.freq10;
    if (d >= -1 && d <= 1) {
      strncpy(buf, s.name, len-1); buf[len-1]='\0'; return;
    }
  }
  strncpy(buf, "MANUAL TUNE", len-1); buf[len-1]='\0';
}

```

```
// ==================== RDA5807M DRIVER ====================
static void rdaWrite() {
  Wire.beginTransmission(RDA_ADDR);
  for (uint8_t r=2; r<=7; r++) {
    Wire.write((uint8_t)(reg[r]>>8));
    Wire.write((uint8_t)(reg[r]&0xFF));
  }
  Wire.endTransmission();
}
static uint16_t rdaRead0A() {
  Wire.requestFrom((uint8_t)RDA_ADDR, (uint8_t)4);
  uint8_t a=Wire.read(), b=Wire.read();
  uint8_t c=Wire.read(); Wire.read();
  sigLevel = constrain((c>>1)/4, 0, 15);
  return ((uint16_t)a<<8)|b;
}
static void rdaInit() {
  reg[2]=0xC001; reg[3]=0; reg[4]=0x0A00;
  reg[5]=(0x88A0&0xFFF0)|(volume&0x0F);
  reg[6]=0; reg[7]=0;
  rdaWrite(); delay(200);
}
static void rdaTune(uint16_t f) {
  uint16_t ch = f-870;
  reg[3]=(ch<<6)|(1u<<4); rdaWrite(); delay(80);
  reg[3]&=~(1u<<4);       rdaWrite();
}
static void rdaSetVol(uint8_t v) {
  v=constrain(v,0,15);
  reg[5]=(reg[5]&0xFFF0)|v; rdaWrite();
}
static bool rdaSeek(bool up) {
  if (up) reg[2]|=(1u<<9); else reg[2]&=~(1u<<9);
  reg[2]|=(1u<<8); rdaWrite(); delay(200);
  uint16_t st=0;
  for (uint8_t i=0; i<80; i++) {
    st=rdaRead0A(); if (st&(1u<<14)) break; delay(50);
  }
  reg[2]&=~(1u<<8); rdaWrite(); delay(50); rdaRead0A();
  if (st&(1u<<13)) return false;
  freq10=constrain(870+(st&0x03FF), FREQ_MIN_10, FREQ_MAX_10);
  return true;
}

// ==================== ENCODER ISRs ====================
void isrEncR() { encRDelta += digitalRead(PIN_ENC_R_DT)?-1:1; }
void isrEncL() { encLDelta += digitalRead(PIN_ENC_L_DT)?-1:1; }

// ==================== BATTERY ====================
int readBattery() {
  long s=0;
  for (uint8_t i=0; i<16; i++) s+=analogRead(PIN_BAT_ADC);
  float v=(s/16.0)*(3.3/1023.0)*2.0;
  if (v>=4.20) return 100; if (v>=4.10) return 90;
  if (v>=4.00) return 80;  if (v>=3.90) return 60;
  if (v>=3.80) return 40;  if (v>=3.70) return 30;
  if (v>=3.60) return 15;  return 0;
}

// ==================== EEPROM ====================
void loadSettings() {
  if (EEPROM.read(EE_MAGIC_ADDR)==EE_MAGIC_VAL) {
    uint8_t lo=EEPROM.read(EE_FREQ_ADDR);
```

```
    uint8_t hi=EEPROM.read(EE_FREQ_ADDR+1);
    freq10=constrain(((uint16_t)hi<<8)|lo, FREQ_MIN_10,FREQ_MAX_10);
    volume=constrain(EEPROM.read(EE_VOL_ADDR), 0, 15);
  }
}
void saveSettings() {
  EEPROM.update(EE_MAGIC_ADDR, EE_MAGIC_VAL);
  EEPROM.update(EE_FREQ_ADDR,   (uint8_t)(freq10&0xFF));
  EEPROM.update(EE_FREQ_ADDR+1, (uint8_t)(freq10>>8));
  EEPROM.update(EE_VOL_ADDR, volume);
}

// ==================== DISPLAY ====================
static uint8_t eqH[20];
static uint16_t rng = 0xACE1;  // LFSR seed
static void advanceEq() {
  for (uint8_t i=0; i<20; i++) {
    rng^=rng<<7; rng^=rng>>9; rng^=rng<<8;
    eqH[i] = 2+(rng&0x07);
  }
}

void drawBoot() {
  oled.clearBuffer();
  oled.setFont(u8g2_font_ncenB14_tr);
  uint16_t w=oled.getStrWidth("FM RADIO");
  oled.drawStr((128-w)/2, 28, "FM RADIO");
  oled.setFont(u8g2_font_ncenB10_tr);
  w=oled.getStrWidth("EE2050");
  oled.drawStr((128-w)/2, 52, "EE2050");
  oled.sendBuffer();
}

void drawMain() {
  oled.clearBuffer();
  oled.setFont(u8g2_font_5x7_tr);

  // Signal bars (top-left, 4 bars)
  int bars=sigLevel/4;
  for (int i=0; i<4; i++) {
    uint8_t h=3+i*2, bx=1+i*4, by=10-h;
    if (i<bars) oled.drawBox(bx,by,3,h);
    else        oled.drawFrame(bx,by,3,h);
  }

  // Battery icon (top-right)
  char tmp[8]; snprintf(tmp,sizeof(tmp),"%d%%",batPercent);
  uint8_t tw=oled.getStrWidth(tmp), tx=128-tw-2;
  oled.drawStr(tx, 9, tmp);
  oled.drawFrame(tx-15, 1, 13, 8);
  oled.drawBox(tx-2, 3, 2, 4);
  int bf=(batPercent*11)/100;
  if (bf>0) oled.drawBox(tx-14, 2, bf, 6);

  // Speaker icon (center-top)
  uint8_t sx=53;
  oled.drawBox(sx, 3, 3, 5);
  oled.drawLine(sx+3,2,sx+7,0); oled.drawLine(sx+3,7,sx+7,10);
  oled.drawLine(sx+7,0,sx+7,10);
  if (volume> 0) oled.drawLine(sx+ 9,4,sx+ 9,7);
  if (volume> 5) oled.drawLine(sx+11,3,sx+11,8);
  if (volume>10) oled.drawLine(sx+13,2,sx+13,9);

```

```
  // Volume -- slot-machine tween (ported from Python sim)
  uint8_t vx=sx+15;
  if (volAnimY<11) {
    int8_t oy=9-volAnimY, ny=9-volAnimY+11;
    char olds[4], news[4];
    snprintf(olds,4,"%d",oldVolume); snprintf(news,4,"%d",volume);
    if (oy>=0 && oy<=10) oled.drawStr(vx, oy, olds);
    if (ny>=0 && ny<=10) oled.drawStr(vx, ny, news);
  } else {
    snprintf(tmp,sizeof(tmp),"%d",volume);
    oled.drawStr(vx, 9, tmp);
  }

  // Frequency (large, centred)
  oled.setFont(u8g2_font_ncenB14_tr);
  snprintf(tmp,sizeof(tmp),"%d.%d",freq10/10,freq10%10);
  uint16_t fw=oled.getStrWidth(tmp);
  oled.drawStr((128-fw)/2, 26, tmp);
  oled.setFont(u8g2_font_5x7_tr);
  oled.drawStr((128-fw)/2+fw+2, 17, "MHz");

  // Station name with SCROLL (ported from Python sim)
  oled.setFont(u8g2_font_7x14B_tr);
  char stn[16]; getStationName(freq10, stn, sizeof(stn));
  uint16_t nw=oled.getStrWidth(stn);
  if (nw<=126) oled.drawStr((128-nw)/2, 40, stn);
  else         oled.drawStr(4-scrollX, 40, stn);

  // Equalizer bars (bottom)
  oled.setFont(NULL);
  for (uint8_t i=0; i<20; i++)
    oled.drawBox(4+i*6, 63-eqH[i], 5, eqH[i]);

  // Status line (HP + ST/MO)
  oled.setFont(u8g2_font_5x7_tr);
  if (hpPlugged) oled.drawStr(0, 63, "HP");
  uint16_t st=rdaRead0A();
  oled.drawStr(12, 63, (st&(1u<<10))?"ST":"MO");

  oled.sendBuffer();
}

void drawSeek() {
  oled.clearBuffer();
  oled.setFont(u8g2_font_7x14B_tr);
  uint16_t w=oled.getStrWidth("Seeking...");
  oled.drawStr((128-w)/2, 36, "Seeking...");
  static uint8_t scanX=0; scanX=(scanX+4)%108;
  oled.drawLine(10,50,118,50);
  oled.drawLine(10+scanX,46,10+scanX,54);
  oled.sendBuffer();
}

// ==================== SETUP ====================
void setup() {
  pinMode(PIN_ENC_R_CLK, INPUT_PULLUP);
  pinMode(PIN_ENC_L_CLK, INPUT_PULLUP);
  pinMode(PIN_ENC_R_DT,  INPUT_PULLUP);
  pinMode(PIN_ENC_L_DT,  INPUT_PULLUP);
  pinMode(PIN_ENC_R_SW,  INPUT_PULLUP);
  pinMode(PIN_ENC_L_SW,  INPUT_PULLUP);
  pinMode(PIN_HP_DET,    INPUT_PULLUP);
  pinMode(PIN_AMP_EN,    OUTPUT);
```

```
  digitalWrite(PIN_AMP_EN, LOW);

  loadSettings(); oldVolume=volume;
  Wire.begin(); oled.begin();
  drawBoot(); bootTime=millis(); booting=true;
  rdaInit(); rdaSetVol(0); rdaTune(freq10);  // start at vol 0 to suppress boot pop
  prevFreq10=freq10; advanceEq();

  attachInterrupt(digitalPinToInterrupt(PIN_ENC_R_CLK),isrEncR,FALLING);
  attachInterrupt(digitalPinToInterrupt(PIN_ENC_L_CLK),isrEncL,FALLING);
  delay(500);  // let amp + DAC settle before enabling speaker
  hpPlugged=!digitalRead(PIN_HP_DET);
  digitalWrite(PIN_AMP_EN, hpPlugged?LOW:HIGH);

  // Ramp volume from 0 to saved value to prevent pop
  for (uint8_t v = 1; v <= volume; v++) {
    rdaSetVol(v);
    delay(30);
  }
}

// ==================== MAIN LOOP ====================
void loop() {
  uint32_t now=millis();
  if (booting && (now-bootTime>2000)) booting=false;

  // Right encoder: tuning (atomic read of ISR variable)
  noInterrupts();
  int8_t rD=encRDelta; encRDelta=0;
  interrupts();
  if (rD && !booting) {
    int16_t f=(int16_t)freq10+rD;
    if (f>FREQ_MAX_10) f=FREQ_MIN_10;
    if (f<FREQ_MIN_10) f=FREQ_MAX_10;
    freq10=f; rdaTune(freq10);
    lastChange=now; needSave=true;
  }

  // Left encoder: volume (atomic read of ISR variable)
  noInterrupts();
  int8_t lD=encLDelta; encLDelta=0;
  interrupts();
  if (lD && !booting) {
    uint8_t nv=constrain((int)volume+lD, 0, 15);
    if (nv!=volume) {
      oldVolume=volume; volume=nv;
      volAnimY=0;
      rdaSetVol(volume);
      lastChange=now; needSave=true;
    }
  }

  // Right button: Seek UP + tactile flash
  if (!digitalRead(PIN_ENC_R_SW) && (now-lastBtnR>300)) {
    lastBtnR=now; flashPending=true;
    drawSeek(); rdaSeek(true);
    scrollState=SCR_WAIT_START; scrollX=0;
    lastChange=now; needSave=true;
  }

  // Left button: Seek DOWN + tactile flash
  if (!digitalRead(PIN_ENC_L_SW) && (now-lastBtnL>300)) {
    lastBtnL=now; flashPending=true;
```

```
    drawSeek(); rdaSeek(false);
    scrollState=SCR_WAIT_START; scrollX=0;
    lastChange=now; needSave=true;
  }

  // Reset scroll on frequency change
  if (freq10!=prevFreq10) {
    prevFreq10=freq10;
    scrollState=SCR_WAIT_START; scrollX=0; scrollTimer=now;
  }

  // Headphone detect
  bool hp=!digitalRead(PIN_HP_DET);
  if (hp!=hpPlugged) {
    hpPlugged=hp;
    digitalWrite(PIN_AMP_EN, hpPlugged?LOW:HIGH);
  }

  // Battery every 5 s
  static uint32_t lastBat=0;
  if (now-lastBat>5000) { lastBat=now; batPercent=readBattery(); }

  // EEPROM save (2 s debounce)
  if (needSave && (now-lastChange>2000)) { saveSettings(); needSave=false; }

  // Display at 100 ms
  if (now-lastDisp<100) return;
  lastDisp=now;

  // Advance animations
  if (volAnimY<11) volAnimY+=2;
  advanceEq();

  // Scroll state machine
  char stn[16]; getStationName(freq10, stn, sizeof(stn));
  oled.setFont(u8g2_font_7x14B_tr);
  uint16_t nw=oled.getStrWidth(stn);
  bool needsScroll=(nw>126);
  if (needsScroll) {
    switch (scrollState) {
      case SCR_WAIT_START:
        if (now-scrollTimer>1500) scrollState=SCR_FWD; break;
      case SCR_IDLE:
        scrollState=SCR_WAIT_START; scrollTimer=now; break;
      case SCR_FWD:
        scrollX+=2;
        if (scrollX>=(int16_t)(nw-120)) {
          scrollX=nw-120;
          scrollState=SCR_PAUSE_END; scrollTimer=now;
        } break;
      case SCR_PAUSE_END:
        if (now-scrollTimer>1500) scrollState=SCR_BACK; break;
      case SCR_BACK:
        scrollX-=2;
        if (scrollX<=0) {
          scrollX=0;
          scrollState=SCR_PAUSE_START; scrollTimer=now;
        } break;
      case SCR_PAUSE_START:
        if (now-scrollTimer>2000) scrollState=SCR_FWD; break;
    }
  } else {
    scrollX=0; scrollState=SCR_IDLE;
```

```
421   }
422
423   // Render frame
424   if (flashPending) {
425     flashPending=false;
426     oled.clearBuffer();
427     oled.setDrawColor(1);
428     oled.drawBox(0,0,128,64);
429     oled.sendBuffer();
430   } else if (booting) {
431     drawBoot();
432   } else {
433     drawMain();
434   }
435 }
```

Listing 1. FM Pocket Radio — ATmega328P Firmware Rev 2 (fm_pocket_radio.ino)

12 Programming the ATmega328P

12.1 Step 1: Burn Bootloader (One-Time Only)

You need an Arduino UNO to act as an ISP programmer.

1. **Prepare Arduino UNO as ISP:**
   - Connect Arduino UNO to your PC via USB
   - In Arduino IDE: File → Examples → 11.ArduinoISP → ArduinoISP
   - Tools → Board → "Arduino Uno"
   - Click Upload (this programs the UNO as an ISP programmer)
2. **Connect UNO to your PCB's ISP header:**

Arduino UNO Pin	PCB ISP Header Pin
D10	RESET (pin 5)
D11 (MOSI)	MOSI (pin 4)
D12 (MISO)	MISO (pin 1)
D13 (SCK)	SCK (pin 3)
5V or 3.3V	VCC (pin 2) — **USE 3.3V!**
GND	GND (pin 6)

WARNING

Use the UNO's 3.3V pin, NOT 5V, to power the PCB during programming. The RDA5807M and other 3.3V components are connected to the same power bus. **Note:** The UNO's 3.3V pin supplies only ∼50 mA (via the FTDI/CH340 regulator). This is sufficient for bare ATmega + passive components during bootloader burn, but **not** for the full radio (OLED + RDA + PAM8403). For full-system ISP programming, use the radio's own battery: turn SW1 ON, then connect only MISO/MOSI/SCK/RESET/GND from the UNO (no VCC wire).

3. **Burn the bootloader:**
   - Tools → Board → "Arduino Pro or Pro Mini"
   - Tools → Processor → "ATmega328P (3.3V, 8 MHz)"

- Tools → Programmer → "Arduino as ISP"
- Tools → Burn Bootloader
- Wait for "Done burning bootloader" message

12.2 Step 2: Upload Firmware (Repeatable)

Firmware is uploaded via the **USB-C port** (J4). The on-board CH340C converts USB to serial automatically.

1. **Plug a USB-C cable** from your PC into J4 on the PCB.

   DANGER

   The radio must be switched ON (SW1) before uploading firmware! The CH340C and ATmega are powered from the +3V3 rail via the HT7333 LDO, which is downstream of the power switch SW1. If SW1 is OFF, the CH340C has no power and the PC will not detect a COM port. Always turn the radio ON first, then plug in USB-C.

2. **In Arduino IDE:**
   - Tools → Board → "Arduino Pro or Pro Mini"
   - Tools → Processor → "ATmega328P (3.3V, 8 MHz)"
   - Tools → Port → (select the CH340 COM port, e.g., COM5)
   - Click Upload
3. The 100 nF cap (C14) between CH340C DTR# and RESET auto-resets the ATmega into bootloader mode. No buttons needed.
4. Upload completes in ∼5 seconds. Unplug the USB-C cable (or leave it in to charge simultaneously).

NOTE

Charging and programming share the same USB-C port. While uploading firmware, the TP4056 also receives 5 V VBUS and charges the battery simultaneously. This is safe — the TP4056 and CH340C share VBUS/GND but have independent data paths.

WARNING

Charge-while-use caveat: The radio **can** operate while USB-C is plugged in (battery charges while radio plays). However, the TP4056's charge termination may be delayed or inaccurate under load, and the battery may not reach a true 4.2 V endpoint. For the fastest, most reliable charge, **turn the radio OFF** (SW1) while charging.

12.3 Troubleshooting Programming

Problem	Solution
"avrdude: stk500_recv(): programmer is not responding"	Check wiring. Ensure MISO/MOSI/SCK/RESET are correct. Ensure UNO has ArduinoISP sketch uploaded.
"avrdude: Yikes! Invalid device signature"	Wrong board selected, or chip is not powered. Check 3.3V supply.

Problem	Solution
USB-C not detected as COM port	Install CH340 driver from WCH website. Windows 10/11 usually auto-detects. Try a different USB-C cable (data-capable, not charge-only). **Ensure the radio is switched ON (SW1)** — the CH340C is powered via the +3V3 rail through SW1.
Upload via USB-C not working	Bootloader not burned. Re-burn via ISP. Verify C14 (100 nF) connects CH340C DTR# to RESET.
Upload times out	TX/RX crossed? CH340C TXD goes to ATmega RX (D0); CH340C RXD goes to ATmega TX (D1).

13 Assembly Guide

13.1 If PCB Shop Pre-Solders

This design uses a **mix of SMD and through-hole**. The university workshop solders the SMD components; you complete the through-hole parts and modules at home.

Workshop solders (SMD — requires hot air station or reflow):

1. PAM8403 (SOP-16) — Class-D amplifier
2. CH340C (SOP-16) — USB-to-serial
3. TP4056 (SOP-8) — LiPo charger
4. HT7333 (SOT-89) — 3.3 V LDO
5. DW01A (SOT-23) + FS8205A (SOT-23-6) — battery protection
6. USB-C receptacle (SMD)
7. LED2 (red, 0805) — CHG indicator
8. LED3 (green, 0805) — STDBY indicator

You solder at home (through-hole):

1. ATmega328P-PU (DIP-28) — in socket or direct solder
2. Crystal Y1 + load caps C3a/C3b (22 pF)
3. All resistors R1–R8, R14 (axial through-hole)
4. Bypass caps C1, C2, C4 (ceramic disc through-hole)
5. Electrolytic caps C6, C7, C9, C10, C16 — observe polarity!
6. Small ceramic cap C8 (100 pF, antenna coupling)
7. Power LED (LED1, 3mm red) — observe polarity!
8. ISP header J3
9. Pin headers: OLED J5, Encoders J6
10. JST battery connector J2
11. 3.5 mm audio jack J1 (PJ-320A)
12. Speaker terminal J8
13. Latching push button SW1
14. Antenna pad J9

You plug in (no soldering required):

1. OLED module into J5 header (check GND/VCC/SCL/SDA orientation)
2. RDA5807M module (solder flat castellated pads to PCB footprint)
3. LiPo battery JST PH connector to J2

4. USB-C cable for charging and programming (then unplug)

13.2 If Hand-Soldering Everything

Order of soldering (lowest profile first, modules last):

1. Resistors R1–R8, R14 (lay flat, axial through-hole)
2. Low-profile through-hole: ceramic capacitors, crystal
3. IC socket (if using DIP-28 ATmega328P — **do not insert chip yet**)
4. Taller components: electrolytic capacitors (observe polarity!), LEDs (observe polarity!)
5. Connectors: 3.5 mm jack, JST battery connector, speaker pads
6. Push button (SW1)
7. Last: insert ATmega328P into socket (align notch/pin 1)

13.3 Module Installation

1. **RDA5807M module:** Solder directly to PCB footprint (castellated pads at 1.27 mm pitch). Use flux and fine-tipped iron. Or solder header pins to the module and plug into matching PCB headers.
2. **SSD1306 OLED:** Plug into 4-pin header J5. Check pin order: GND, VCC, SCL, SDA.

   WARNING

   OLED module pin order varies by manufacturer! Some modules use GND–VCC–SCL–SDA, others use VCC–GND–SCL–SDA or even SDA–SCL–VCC–GND. **Always verify your specific module's pin order** before plugging in. Reversing VCC and GND **will damage the OLED.** Label J5 on the PCB silkscreen with the expected pin order.
3. **Encoders:** Connect via 5-pin headers (J6). Route wires to enclosure mounting holes.

13.4 Final Assembly

1. Plug in battery (JST connector)
2. Do NOT close the enclosure yet
3. Proceed to testing (Section 14)
4. After all tests pass, mount PCB in enclosure

14 Testing & Verification

Follow these tests in order. Do not skip steps.

14.1 Test 1: Visual Inspection

- ☐ Check all solder joints for bridges (shorts) under magnification
- ☐ Verify ATmega328P orientation (notch at pin 1)
- ☐ Verify electrolytic cap polarity (stripe = negative)
- ☐ Verify LED polarity (flat side = cathode)

☐ Verify SMD solder joints: TP4056, CH340C, PAM8403, HT7333, DW01A, USB-C connector (no bridges, no cold joints)

14.2 Test 2: Power Supply (Before Inserting ICs)

If using a DIP socket, do this test **without** the ATmega328P inserted.

1. Connect battery via JST connector
2. Press power switch ON

☐ Measure VBAT: should be 3.0–4.2 V

☐ Measure +3V3 rail: should be 3.3 V (±0.1 V)

☐ Power LED lights up

☐ No components getting hot

3. Press power switch OFF

☐ All voltages drop to 0 V

14.3 Test 3: TP4056 Charging via USB-C

1. Plug a USB-C cable from a 5 V charger (or PC) into J4

☐ Red CHG LED (LED2 on PCB) lights up — battery is charging

☐ Measure BAT pad: should be charging voltage (3.0–4.2 V rising)

2. Wait until green STDBY LED (LED3 on PCB) lights up — battery full

☐ No components getting excessively hot (warm is OK, hot is not)

14.4 Test 4: MCU Boot

1. Insert ATmega328P (if DIP socket)
2. Burn bootloader (Section 12)
3. Upload firmware
4. Power on

☐ Boot screen appears on OLED (“FM RADIO”, “EE2050”) for 2 seconds

☐ Main screen appears after boot

14.5 Test 5: I2C Scan

Upload a basic I2C scanner sketch first (before the radio firmware):

```
#include <Wire.h>
void setup() {
  Wire.begin();
  Serial.begin(9600);
  for (byte a = 1; a < 127; a++) {
    Wire.beginTransmission(a);
    if (Wire.endTransmission() == 0) {
      Serial.print("Found: 0x");
      Serial.println(a, HEX);
    }
  }
}
```

```
void loop() {}
```

- ☐ Serial output shows: `Found: 0x10` (RDA5807M)
- ☐ Serial output shows: `Found: 0x3C` (SSD1306)

NOTE

RDA5807M address note: Most RDA5807M modules respond at `0x10` (sequential write mode). Some modules may also appear at `0x11` (random-access mode). If you see `0x11` instead of (or in addition to) `0x10`, the module is working correctly — the firmware uses the 0x10 address for sequential register writes.

14.6 Test 6: FM Radio Reception

1. Plug in headphones (they are the antenna)
2. Rotate right encoder to tune

- ☐ Audio plays through headphones
- ☐ Station names appear on display when tuned to known frequencies
- ☐ Signal bars change with frequency

14.7 Test 7: Speaker Output

1. Unplug headphones

- ☐ Speaker starts playing (MCU pulls D9/AMP_EN HIGH, PAM8403 unmutes)
- ☐ Rotate left encoder — volume changes (0–15 on display)
- ☐ Press right encoder — seek UP finds next station
- ☐ Press left encoder — seek DOWN finds previous station

14.8 Test 8: Headphone Auto-Mute

1. While speaker is playing, plug in headphones

- ☐ Speaker mutes immediately
- ☐ Audio plays through headphones
- ☐ Display shows “[HP]” indicator

2. Unplug headphones

- ☐ Speaker resumes
- ☐ Display shows “[SPK]”

14.9 Test 9: EEPROM Persistence

1. Tune to a specific frequency (e.g., 99.5 MHz)
2. Set volume to a specific level (e.g., 12)
3. Wait 3 seconds (EEPROM save debounce is 2s)
4. Power off, wait 5 seconds, power on

- ☐ Radio tunes to 99.5 MHz automatically

☐ Volume is 12

14.10 Test 10: Battery Level

☐ Battery percentage displays on screen

☐ Percentage drops as battery discharges

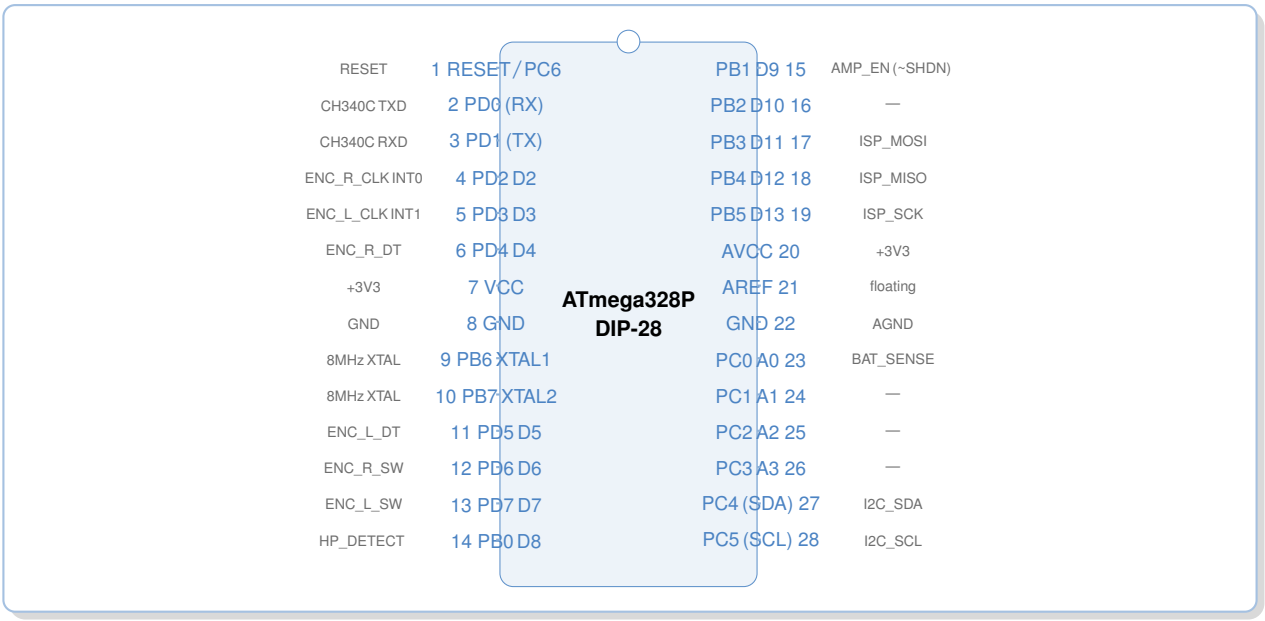
☐ At full charge (∼4.2 V): shows ∼100%

15 Safety Precautions

DANGER

1. **LiPo Battery:**
   - Never short-circuit the battery
   - Never charge unattended on flammable surfaces
   - Never puncture, crush, or expose to heat >60°C
   - Use a protected cell (has built-in over-discharge protection)
   - If the battery puffs/swells, dispose of it safely immediately
2. **3.3 V Devices:**
   - NEVER connect 5 V to the RDA5807M VCC — instant destruction
   - NEVER connect 5 V to I2C pull-ups
   - When programming via UNO ISP, power from UNO's 3.3 V pin, NOT 5 V
3. **Soldering:**
   - Work in ventilated area (flux fumes are harmful)
   - Use lead-free solder if possible
   - Do not touch hot iron tip or recently soldered joints
   - Place decoupling caps within 2 mm of IC power pins
   - Check electrolytic cap and LED polarity before soldering
4. **ESD (Electrostatic Discharge):**
   - Touch a grounded metal object before handling ICs
   - Store ICs in anti-static bags until ready to solder
   - The RDA5807M and ATmega328P are sensitive to ESD

A ATmega328P-PU (DIP-28) Full Pinout



B RDA5807M Key Registers

Reg	Init Value	Key Bits
0x02	0xC001	[15] DHIZ=1 (audio high-Z disable), [14] DMUTE=1 (unmute), [0] ENABLE=1
0x03	(channel)	[15:6] CHAN (frequency channel), [4] TUNE (start tuning), [3:2] BAND=00 (87-108), [1:0] SPACE=00 (100kHz)
0x04	0x0A00	[11] DE=1 (50 μs de-emphasis), [9] SOFTMUTE_EN=1
0x05	0x88Ax	[15:12] INT_MODE, [11:8] SEEKTH=8, [3:0] VOLUME (0–15)
0x0A	(read)	[14] STC (seek/tune complete), [13] SF (seek fail), [9:0] READCHAN
0x0B	(read)	[15:9] RSSI (signal strength, 0–127)

Frequency formula: $\text{Channel} = \frac{\text{Freq(MHz)} - 87.0}{0.1}$ (Band=00, Space=00)

Reverse: $\text{Freq(MHz)} = 87.0 + \text{Channel} \times 0.1$

C KiCad Custom Library Quick Reference

Components you must import or create (not in KiCad default libraries):

Component	Source	Import Method
PAM8403 (SOP-16)	SnapEDA	SnapEDA download; footprint `Package_SO:SOP-16_4.4x10.4mm_P1.27mm`

Component	Source	Import Method
CH340C (SOP-16)	SnapEDA or LCSC	SnapEDA download; footprint `Package_SO:SOP-16_4.4x10.4mm_P1.27mm`
TP4056 (SOP-8)	SnapEDA	SnapEDA download; footprint `Package_SO:SOP-8_3.9x4.9mm_P1.27mm`
HT7333 (SOT-89)	SnapEDA	SnapEDA download; footprint `Package_TO_SOT_SMD:SOT-89-3`
DW01A (SOT-23)	SnapEDA	SnapEDA download; footprint `Package_TO_SOT_SMD:SOT-23`
FS8205A (SOT-23-6)	SnapEDA	SnapEDA download; footprint `Package_TO_SOT_SMD:SOT-23-6`
USB-C receptacle	SnapEDA or LCSC	Depends on exact part; 16-pin SMD footprint
RRD-102 V2.0 module	Create manually	Custom symbol (10 pins) + custom footprint (11×11mm, 1.27mm pitch)
SSD1306 OLED module	Create manually	Simple 4-pin symbol + use built-in 1×4 header footprint
3.5mm Audio Jack	KiCad built-in	`Connector_Audio:AudioJack3_SwitchT`
Rotary Encoder	KiCad built-in	`Device:RotaryEncoder_Switch`
ATmega328P-PU	KiCad built-in	`MCU_Microchip_ATmega:ATmega328P-PU`
Crystal	KiCad built-in	`Device:Crystal`

End of document. Good luck with your pocket radio!